Name: Bhushan Bhaskar Sagar

……..

Problems with Hadoop Map-Reduce?

1. Batch Processing: Hadoop and MapReduce are designed for batch processing, making them unfit for real-time or near real-time processing such as streaming data.

2. Complexity: Hadoop has a steep learning curve, and its setup, configuration, and maintenance can be complex and time-consuming.

3. Data Movement: Hadoop's architecture can lead to inefficiencies and network congestion when dealing with smaller data sets.

4. Fault Tolerance: While Hadoop has data replication for fault tolerance, it can lead to inefficient storage use and doesn't cover application-level failures.

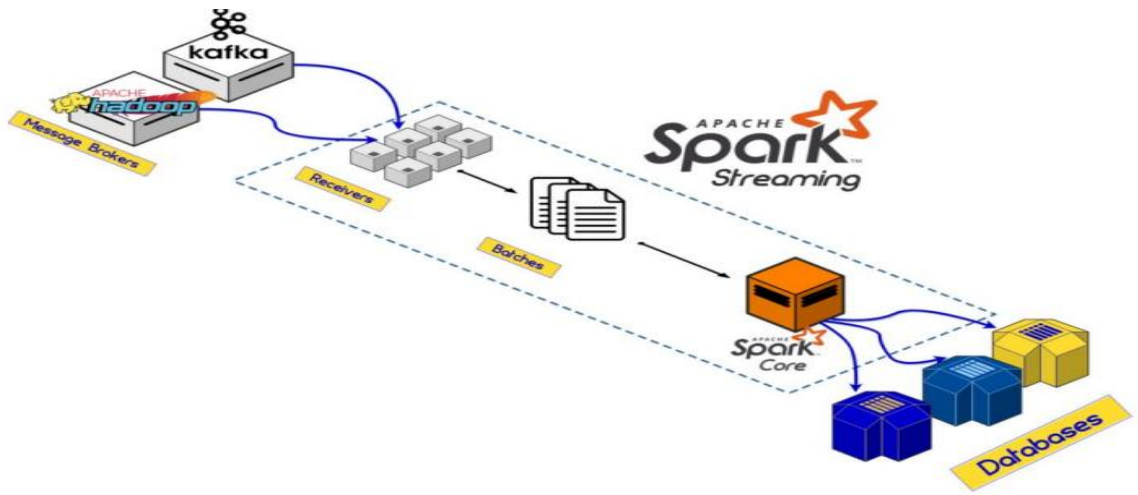
5. No Support for Interactive Processing: MapReduce doesn't support interactive processing, making it unsuitable for tasks needing back-and-forth communication.

6. Not Optimal for Small Files: Hadoop is less effective with many small files, as it's designed to handle large data files.

……..

What is Apache Spark?

Apache Spark is an open-source, distributed computing system designed for big data processing and analytics. It provides an interface for programming entire clusters with implicit data parallelism and fault tolerance. Spark is known for its speed, ease of use, and versatility in handling multiple types of data workloads, including batch processing, real-time data streaming, machine learning, and interactive queries.



Features Of Spark:

1. Speed: Compared to Hadoop MapReduce, Spark can execute large-scale data processing up to 100 times faster. This speed is achieved by leveraging controlled partitioning.

2. Powerful Caching: Spark's user-friendly programming layer delivers impressive caching and disk persistence capabilities.

3. Deployment: Spark offers versatile deployment options, including through Mesos, Hadoop via YARN, or its own cluster manager.

4. Real-Time Processing: Thanks to in-memory computation, Spark facilitates real-time computation and offers low latency.

5. Polyglot: Spark provides high-level APIs in several languages - Java, Scala, Python, and R, allowing code to be written in any of these. It also offers a shell in Scala and Python.

6. Scalability: Spark's design is inherently scalable, capable of handling and processing large amounts of data by distributing tasks across multiple nodes in a cluster.

…….

Spark Ecosystem:

1. Spark Core Engine: The foundation of the entire Spark ecosystem, the Spark Core, handles essential functions such as task scheduling, monitoring, and basic I/O operations. It also provides the core programming abstraction, Resilient Distributed Datasets (RDDs).

2. Cluster Management: Spark's versatility allows for cluster management by multiple tools, including Hadoop YARN, Apache

Mesos, or Spark's built-in standalone cluster manager. This flexibility accommodates varying requirements and operational contexts.

3. Library: The Spark ecosystem includes a rich set of libraries:

a. Spark SQL allows SQL-like queries on RDDs or data from external sources, integrating relational processing with Spark's functional programming API.

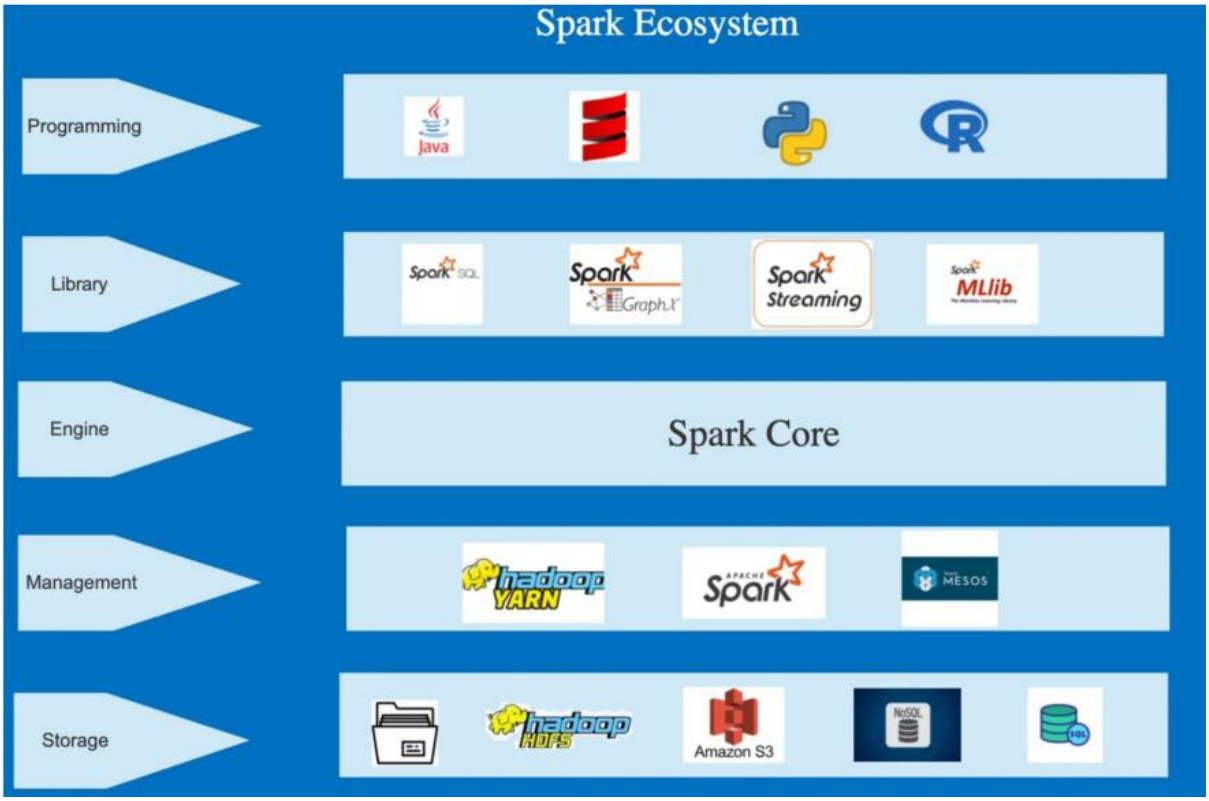
b. Spark MLlib is a library for machine learning that provides various algorithms and utilities.

c. Spark GraphX allows for the construction and computation on graphs, facilitating advanced data visualization and graph computation.

d. Spark Streaming makes it easy to build scalable, high-throughput, fault-tolerant streaming applications that can handle live data streams alongside batch processing.

4. Polyglot Programming: Spark supports programming in multiple languages including Python, Java, Scala, and R. This broad language support makes Spark accessible to a wide range of developers and data scientists.

5. Storage Flexibility: Spark can interface with a variety of storage systems, including HDFS, Amazon S3, local filesystems, and more. It also supports interfacing with both SQL and NoSQL databases, providing broad flexibility for various data storage and processing needs.



……

RDD in Spark:

RDDs are the building blocks of any Spark application.

RDDs Stands for:

- Resilient: Fault tolerant and is capable of rebuilding data on failure
- Distributed: Distributed data among the multiple nodes in a cluster
- Dataset: Collection of partitioned data with values

RDDs have been a fundamental building block in Spark, but in newer versions of Spark (2.0 and later.

Resilient Distributed Datasets (RDD) are a core abstraction in Apache Spark. Here are some key points about RDDs and their properties:

1. Fundamental Data Structure: RDD is the fundamental data structure of Spark, which allows it to efficiently operate on large-scale data across a distributed environment.
2. Immutability: Once an RDD is created, it cannot be changed. Any transformation applied to an RDD creates a new RDD, leaving the original one untouched. RDDs are immutable, which means their content cannot be changed once they are created. If you want to transform an RDD, you create a new RDD from the existing one through transformations.
3. Resilience: RDDs are fault-tolerant, meaning they can recover from node failures. This resilience is provided through a feature known as lineage, a record of all the transformations applied to the base data. RDDs are resilient, meaning they can recover from node failures. Spark keeps track of the transformations applied to an RDD and can recompute lost data when necessary. This resilience is achieved through lineage information, which records the sequence of transformations that created an RDD.
4. Lazy Evaluation: RDDs follow a lazy evaluation approach, meaning transformations on RDDs are not executed immediately, but computed only when an action (like count, collect) is performed. This leads to optimized computation. Spark uses lazy evaluation, which means that transformations on RDDs are not executed immediately when called. Instead, Spark builds a logical execution plan (DAG) and only applies the transformations when an action is invoked. This optimization helps minimize unnecessary computation.
5. Partitioning: RDDs are partitioned across nodes in the cluster, allowing for parallel computation on separate portions of the dataset.
6. In-Memory Computation: RDDs can be stored in the memory of worker nodes, making them readily available for repeated access, and thereby speeding up computations.
7. Distributed Nature: RDDs can be processed in parallel across a Spark cluster, contributing to the overall speed and scalability of Spark. RDDs are distributed across a cluster of machines. Each RDD is divided into partitions, with each partition residing on a separate node in the cluster. This distribution enables parallel processing and fault tolerance.
8. Persistence: Users can manually persist an RDD in memory, allowing it to be reused across parallel operations. This is useful for iterative algorithms and fast interactive use.

9. Operations: Two types of operations can be performed on RDDs - transformations (which create a new RDD) and actions (which return a value to the driver program or write data to an external storage system)
    - Transformations: These are operations that produce a new RDD from one or more existing RDDs. Examples include map, filter, and reduceByKey. Transformations are lazy and only record the operation in the lineage.
    - Actions: These are operations that trigger the execution of transformations and return a value to the driver program or write data to an external storage system. Examples include count, collect, and saveAsTextFile. Actions force Spark to execute the transformations required to produce the result.
10. Data Sources: RDDs can be created from various data sources, including HDFS (Hadoop Distributed File System), local file systems, Cassandra, HBase, and more.

……

How Spark Perform Data Partitioning?

Data partitioning in Apache Spark is the process of dividing large datasets into smaller, more manageable pieces called partitions. Each partition is a logical division of the data that can be processed independently by a separate task running on a cluster node. Data partitioning is a fundamental concept in Spark's distributed data processing model and is crucial for achieving parallelism and scalability in data processing.

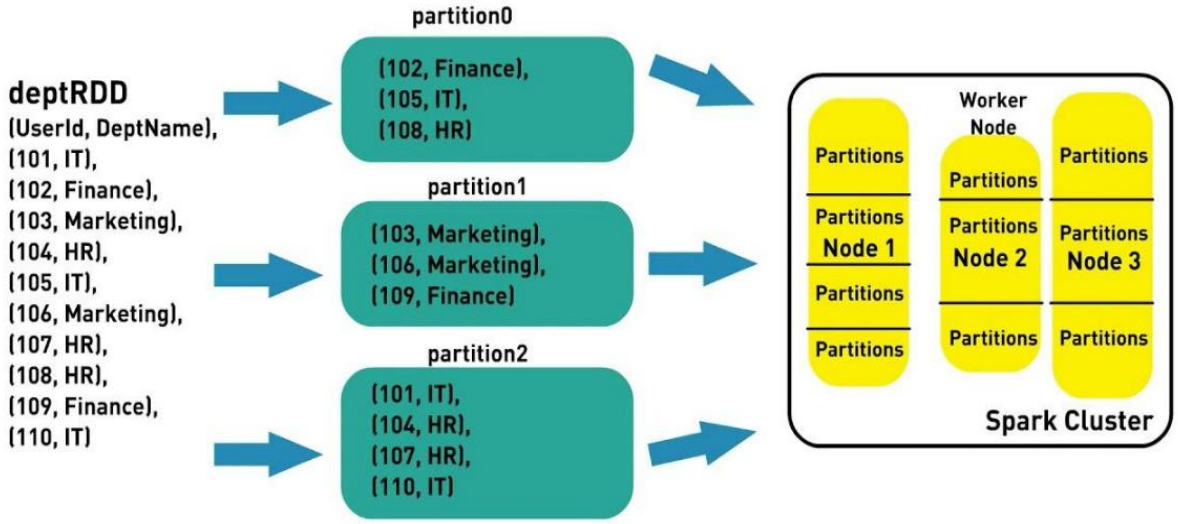
1. Data Partitioning: Apache Spark partitions data into logical chunks during reading from sources like HDFS, S3, etc

2. Data Distribution: These partitions are distributed across the Spark cluster nodes, allowing for parallel processing.

3. Custom Partitioning: Users can control data partitioning using Spark's repartition(), coalesce() and partitionBy() methods, optimizing data locality or skewness.

When Apache Spark reads data from a file on HDFS or S3, the number of partitions is determined by the size of the data and the default block size of the file system. In general, each partition corresponds to a block in HDFS or an object in S3.

For example, if HDFS is configured with a block size of 128MB and you have a 1GB file, it would be divided into 8 blocks in HDFS. Therefore, when Spark reads this file, it would create 8 partitions, each corresponding to a block.



.....

Transformation in Spark:

Apache Spark, "transformation" refers to the process of creating a new Resilient Distributed Dataset (RDD) or a DataFrame from an existing one. Spark provides a set of transformation operations that allow you to manipulate and process data in a distributed and parallel manner. Transformations are executed lazily, which means that they don't compute their results immediately; instead, they build a logical execution plan that is only executed when an action is triggered.

In Spark, a transformation is an operation applied on an RDD (Resilient Distributed Dataset) or DataFrame/Dataset to create a new RDD or DataFrame/Dataset.

Transformations in Spark are categorized into two types: narrow and wide transformations.

Narrow Transformations: In these transformations, all elements that are required to compute the records in a single partition live in the same partition of the parent RDD. Data doesn't need to be shuffled across partitions.

Examples include:

● map(): Applies a function to each element in the RDD and outputs a new RDD.

● filter(): Creates a new RDD by selecting only the elements of the original RDD that pass a function's condition.

● flatMap(): Function in Spark applies a function to each element of an RDD, then flattens the multiple outputs into a single RDD.

● sample(): Create a sample dataset from the original data.

Wide Transformations: These transformations will have input data from multiple partitions. This typically involves shuffling all the data across multiple partitions.

Examples include:

● groupByKey(): Groups all the values of each key in the RDD into a single sequence.

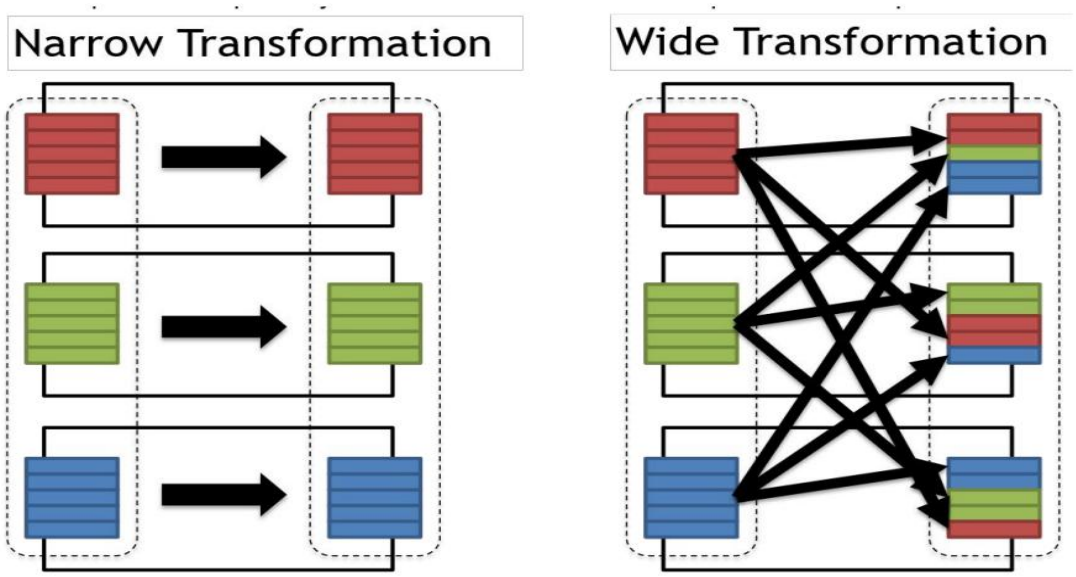
● reduceByKey(): Performs a reduction operation for each key in the RDD.

● join(): Joins two RDDs based on a common key, similar to the SQL JOIN operation.

● distinct(): Remove duplicates in the RDD.

● coalesce(): Decreases the number of partitions in the RDD.

● repartition(): Increases the number of partitions in the RDD.



……

Action in Spark:

Apache Spark, "actions" are operations that trigger the execution of transformations on the underlying distributed data and return results to the driver program or write data to external storage. Unlike transformations, which are lazily evaluated and build a logical execution plan, actions are the operations that compute and produce a result. Actions are what kickstart the distributed data processing tasks in Spark.

Actions in Apache Spark are operations that provide non-RDD values; they return a final value to the driver program or write data to an external system. Actions trigger the execution of the transformation operations accumulated in the Directed Acyclic Graph (DAG).

Here are some of the commonly used actions in Spark:

● Collect: collect() returns all the elements of the RDD as an array to the driver program. This can be useful for testing and debugging, but be careful with large datasets to avoid out-of-memory errors.

● Count: count() returns the number of elements in the RDD.

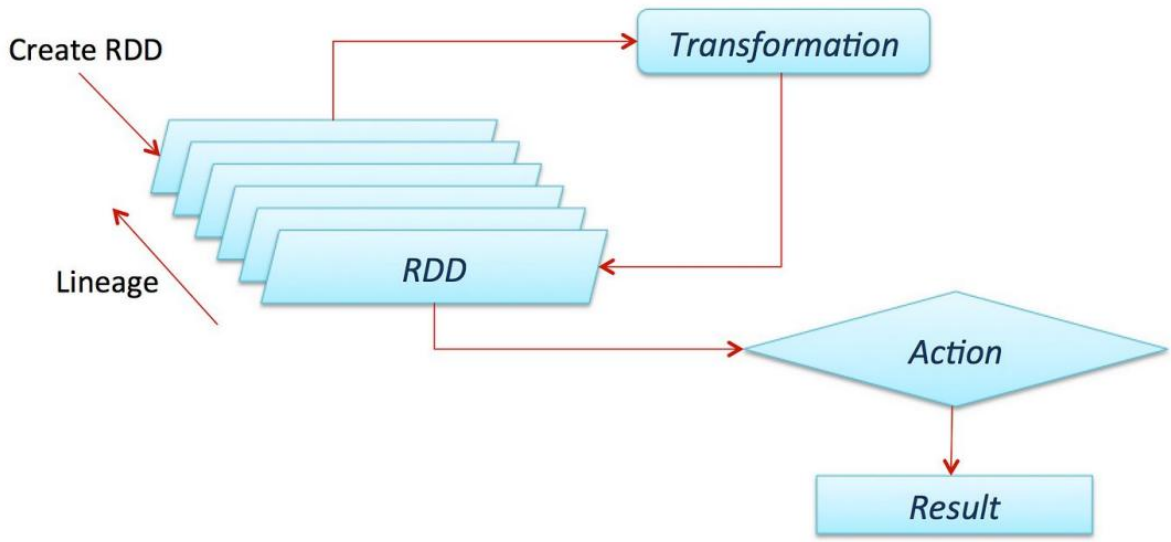
● First: first() returns the first element of the RDD.

● Take: take(n) returns the first n elements of the RDD.

● foreach: foreach() is used for performing computations on each element in the RDD.

● SaveAsTextFile: saveAsTextFile() writes the elements of the dataset to a text file (or set of text files) in a specified directory in the local filesystem, HDFS, or any other Hadoop-supported file system.

● SaveAsSequenceFile: This action is used to save RDDs, which consist of key/value pairs, in SequenceFile format.



……

Read & Write operation in Spark are Transformation/Action?

Reading and writing operations in Spark are often viewed as actions, but they're a bit unique. Let's clarify this:

● Read Operation: Reading data in Spark, such as from a file, a database, or another data source, is typically not considered a transformation or an action. Rather, it's an initialization step that creates an RDD or DataFrame/Dataset, upon which transformations and actions are applied. However, this read operation triggers a job execution if it's a "first touch" action, meaning Spark employs lazy evaluation and won't actually load the data until an action necessitates it.

● Write Operation: Writing or saving data in Spark, on the other hand, is considered an action. Functions like saveAsTextFile(), saveAsSequenceFile(), saveAsObjectFile(), or DataFrame/Dataset's write options trigger computation and result in data being written to an external system

……..

Lazy Evaluation in Spark:

Lazy evaluation is an important concept in Apache Spark, a distributed data processing framework. Lazy evaluation means that Spark postpones the execution of transformations on its Resilient Distributed Dataset (RDD) until an action is called. This concept is central to Spark's optimization and fault tolerance mechanisms.

Lazy evaluation in Spark means that the execution doesn't start until an action is triggered. In Spark, transformations are lazily evaluated, meaning that the system records how to compute the new RDD (or DataFrame/Dataset) from the existing one without performing any transformation. The transformations are only actually computed when an action is called and the data is required.

Here's how lazy evaluation works in Spark:

Transformations: Spark provides a set of transformation operations, such as map, filter, reduceByKey, and more, which are applied to RDDs to create new RDDs. When you perform these transformations on an RDD, Spark doesn't immediately execute them. Instead, it builds a logical execution plan (DAG - Directed Acyclic Graph) that represents the sequence of transformations to be applied.

Actions: Actions are operations that trigger the execution of the logical execution plan and initiate the actual computation. Examples of actions include collect, count, saveAsTextFile, and reduce. When you call an action, Spark starts the evaluation process from the top of the execution plan and works its way down through the transformations.

Lazy evaluation has several advantages in Spark:

Optimization: Spark can optimize the execution plan by combining multiple transformations into a single stage or reordering operations for efficiency. This optimization is possible because Spark has a holistic view of the entire execution plan.

Fault tolerance: Lazy evaluation allows Spark to recover from failures by recomputing only the lost data partitions and not re-running the entire transformation sequence.

Reduced I/O: By delaying execution until an action is called, Spark can minimize unnecessary disk I/O, as it knows precisely which data partitions are required to compute the result.

Example:

```
# Create an RDD and apply transformations

data = [1, 2, 3, 4, 5]

rdd = sc.parallelize(data)

mapped_rdd = rdd.map(lambda x: x * 2)

filtered_rdd = mapped_rdd.filter(lambda x: x % 4 == 0)

# Up to this point, no actual computation has occurred

# Perform an action to trigger the execution

result = filtered_rdd.collect()

# At this point, the entire execution plan is executed
```

In the example above, transformations like map and filter are only executed when the collect action is called. Lazy evaluation enables Spark to optimize and execute these transformations in the most efficient way.

Lazy evaluation is one of the key reasons behind Spark's high performance and efficiency in distributed data processing.

……..

Lineage Graph or DAG in Spark:

Apache Spark, the lineage graph, also known as the Directed Acyclic Graph (DAG), is a critical data structure that represents the logical execution plan of transformations applied to a Resilient Distributed Dataset (RDD). The lineage graph is a fundamental concept in Spark's ability to recover from node failures and optimize the execution of distributed data processing jobs.

Spark represents a sequence of transformations on data as a DAG, a concept borrowed from mathematics and computer science. A DAG is a directed graph with no cycles, and it represents a finite set of transformations on data with multiple stages. The nodes of the graph represent the RDDs or DataFrames/Datasets, and the edges represent the transformations or operations applied. Each action on an RDD (or DataFrame/Dataset) triggers the creation of a new DAG. The DAG is optimized by the Catalyst optimizer (in case of DataFrame/Dataset) and then it is sent to the DAG scheduler, which splits the graph into stages of tasks.

Here's how the lineage graph works:

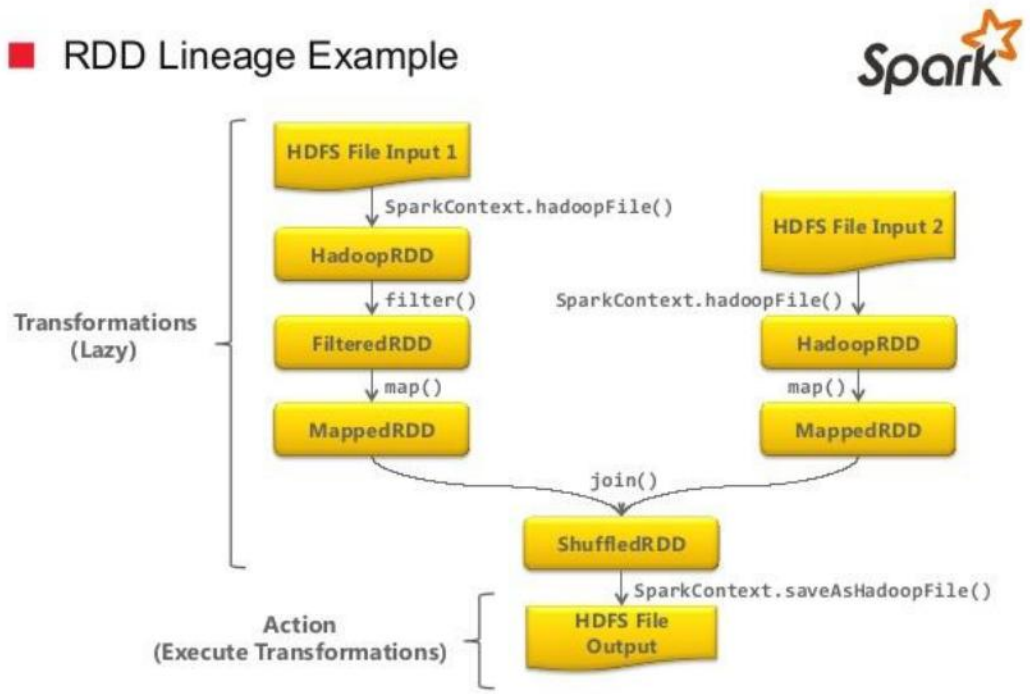
- Transformation Operations: When you perform transformations on an RDD (e.g., map, filter, reduceByKey, etc.), Spark doesn't immediately execute them. Instead, it records these transformations as vertices in the lineage graph.
- Dependencies: Each vertex in the lineage graph includes information about its dependencies on other RDDs. This dependency information creates a directed edge in the lineage graph, representing how data flows from one RDD to another as a result of transformations.
- Actions: When an action (e.g., collect, count, saveAsTextFile, etc.) is called on an RDD, it triggers the execution of the lineage graph. Spark starts from the RDD produced by the initial

data source (e.g., reading from HDFS or other storage systems) and follows the graph to compute the final result.

Key characteristics of the lineage graph (DAG) in Spark:

- Directed Acyclic Graph (DAG): The lineage graph is a directed acyclic graph, which means it has a clear direction of data flow, and it doesn't contain any cycles. This acyclic property is essential for ensuring that data can be recomputed from source data in case of failures.

- Fault Tolerance: One of the primary advantages of the lineage graph is fault tolerance. If a node or task fails during the execution of a Spark job, the lineage graph provides the information needed to recompute only the lost partitions, minimizing the need to reprocess the entire dataset.
- Optimization: The lineage graph enables Spark to optimize the execution plan. It can merge multiple transformations into a single stage, reorder operations for efficiency, and eliminate unnecessary computations based on the logical execution plan represented by the graph.
- Materialization: The lineage graph also keeps track of where data is materialized (cached) in memory or on disk. This information is used to avoid redundant computation by reusing cached data when possible.



……

Job, Stage and Task in Spark:

In Apache Spark, a distributed data processing framework, jobs, stages, and tasks are key concepts that play a vital role in the execution of data processing operations. These concepts help organize and manage the parallel processing of data in a distributed environment efficiently. Here's an overview of each:

1. **Job:**
   A job in Spark represents a single action (like count, collect, save, etc.) from a Spark application. When an action is called on a DataFrame or RDD in the program, a job is created. A job is a full program from start to finish, including reading the initial data, performing transformations, and executing actions. A Spark application can consist of multiple jobs, and each job is independent of the others.
   - A Spark job is the highest-level unit of work in Spark. It represents a complete computation that consists of a sequence of transformations and actions on RDDs.
   - A Spark application can consist of one or more jobs, and each job typically corresponds to one or more actions called on RDDs.
   - A Spark job is submitted to the Spark cluster manager, which divides the job into multiple stages for execution.
2. **Stage:**
   A stage in Spark is a sequence of transformations on an RDD or DataFrame/Dataset that can be performed in a single pass (i.e., without shuffling data around). Spark splits the computation of a job into a series of stages, separated by shuffle boundaries. Each stage represents a sequence of transformations that can be done in a single scan of the data. In essence, a stage is a step in the physical execution plan.
   - A Spark job is divided into one or more stages, based on the data shuffling dependencies between transformations. Stages are created to minimize data shuffling and optimize parallel execution.

   There are two types of stages in Spark: shuffle stages and non-shuffle stages.

   - Shuffle Stages: These stages involve data shuffling, typically when operations like reduceByKey, groupByKey, or join are performed. Data must be reorganized and moved between partitions, which introduces a performance cost.
   - Non-shuffle Stages: These stages involve operations that can be computed independently without data shuffling.Each stage is composed of multiple tasks.
3. **Task:**
   Within each stage, the data is further divided into partitions, and each partition is processed in parallel. A task in Spark corresponds to a single unit of work sent to one executor. So, if you have two stages with two partitions each, Spark will generate four tasks in total - one task per partition per stage. Each task works on a different subset of the data, and the tasks within a stage can be run in parallel.
   - A task is the smallest unit of work in Spark and corresponds to the execution of a single partition of a stage. Tasks are the actual units of parallel execution within Spark.
   - Tasks are created based on the number of partitions in the input RDDs and are executed in parallel across the cluster.
   - For non-shuffle stages, each task processes data independently, so the tasks in a stage can execute in parallel with minimal inter-task communication.

- For shuffle stages, tasks need to exchange data with each other, which can involve data serialization, transfer, and deserialization. This is where much of the performance overhead occurs.
- Spark's task scheduler distributes tasks to worker nodes, and each worker node processes the tasks assigned to it in parallel.

To illustrate these concepts, let's consider an example:

Suppose you have a Spark job that reads a dataset from HDFS, performs several map and filter transformations (non-shuffle stages), and then performs a reduceByKey operation (shuffle stage). The job can be broken down as follows:

Job 1 (Top-level job)

Stage 1 (Non-shuffle stage, e.g., map and filter)

Task 1 (Processes partition 1 of input data)

Task 2 (Processes partition 2 of input data)

...

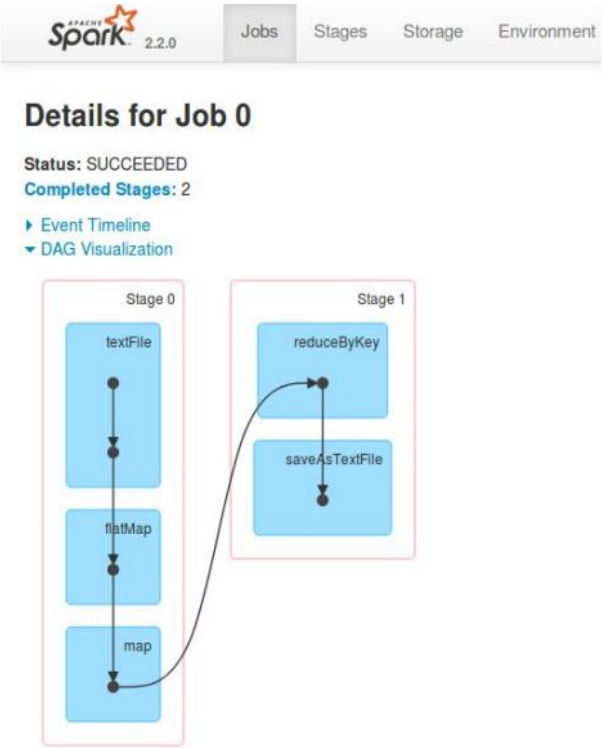
Stage 2 (Shuffle stage, reduceByKey)

Task 3 (Shuffling data, processing after data exchange)

Task 4 (Shuffling data, processing after data exchange)

...

Spark's job scheduler manages the execution of these jobs, stages, and tasks, distributing the work across the cluster's worker nodes, optimizing data movement, and ensuring fault tolerance through lineage tracking.



Example 2:

```
from pyspark.sql import SparkSession

# Initialize Spark

spark = SparkSession.builder \

 .appName("PySpark Application") \

 .getOrCreate()

# Read CSV files

employees = spark.read.csv('employees.csv', inferSchema=True, header=True)

departments = spark.read.csv('departments.csv', inferSchema=True, header=True)

regions = spark.read.csv('regions.csv', inferSchema=True, header=True) # Adding a third DataFrame

# Narrow transformation: Filter

filtered_employees = employees.filter(employees.age > 30)

# Wide transformation: Join

result = filtered_employees.join(departments, filtered_employees.dept_id ==
departments.dept_id)

# Another wide transformation: Join with regions

result_with_regions = result.join(regions, result.region_id == regions.region_id)

# Action: Collect

result_list = result_with_regions.collect()

# Narrow transformation: Select a few columns

selected_data = result_with_regions.select('employee_name', 'department_name', 'region_name')

# Action: Save as CSV

selected_data.write.csv('result.csv')

# Stop Spark

spark.stop()
```

The jobs and their associated stages in the PySpark script example would be as follows:

Job 1:

Triggered by the collect() action.

This job consists of three stages:

● Stage 1: Filter transformation on 'employees' DataFrame.

● Stage 2: Join transformation between 'filtered_employees' and 'departments' DataFrames.

● Stage 3: Join transformation between 'result' and 'regions' DataFrames.

Job 2:

Triggered by the write.csv() action. This job consists of one stage: The select() transformation and the write.csv() action do not require a shuffle and therefore do not trigger a new stage within Job 2.

……

What if our cluster capacity is less than the size of data to be processed?

If your cluster memory capacity is less than the size of the data to be processed, Spark can still handle it by leveraging its ability to perform computations on disk and spilling data from memory to disk when necessary. Let's break down how Spark will handle a 60 GB data load with a 30 GB memory cluster:

● Data Partitioning: When Spark reads a 60 GB file from HDFS, it partitions the data into manageable blocks, according to the Hadoop configuration parameter dfs.blocksize or manually specified partitions. These partitions can be processed independently.

● Loading Data into Memory: Spark will load as many partitions as it can fit into memory. It starts processing these partitions. The size of these partitions is much smaller than the total size of your data (60 GB), allowing Spark to work within the confines of your total memory capacity (30 GB in this case).

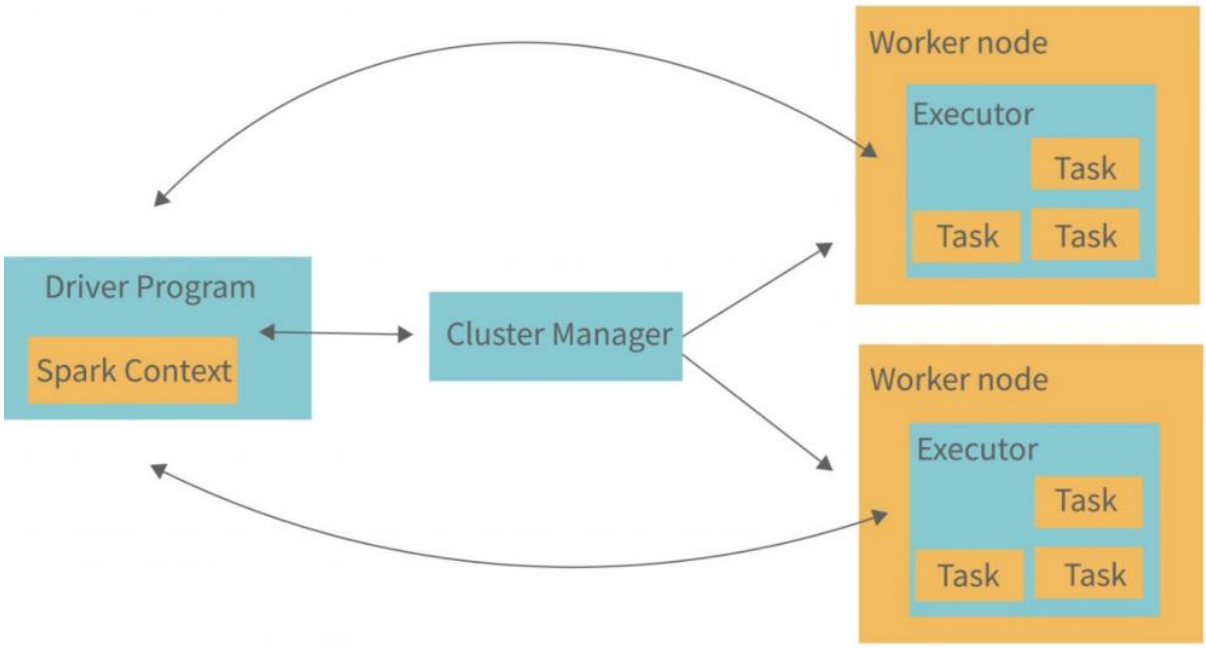
● Spill to Disk: When the memory is full, and Spark needs to load new partitions for processing, it uses a mechanism called "spilling" to free up memory. Spilling means writing data to disk. The spilled data is the intermediate data generated during shuffling operations, which needs to be stored for further stages.

● On-Disk Computation: Spark has the capability to perform computations on data that is stored on disk, not just in memory. Although computations on disk are slower than in memory, it allows Spark to handle datasets that are larger than the total memory capacity.

● Sequential Processing: The stages of the job are processed sequentially, meaning Spark doesn't need to load the entire dataset into memory at once. Only the data required for the current stage needs to be in memory or disk.

……

Spark Architecture & Its components:



● Driver Program: The driver program is the heart of a Spark application. It runs the main() function of an application and is the place where the SparkContext is created. SparkContext is responsible for coordinating and monitoring the execution of tasks. The driver program defines datasets and applies operations (transformations & actions) on them.

- The driver program is the entry point of a Spark application. It runs the main function and creates a SparkContext to coordinate the job.
- The driver program defines the application and breaks it into tasks.

● SparkContext: The SparkContext is the main entry point for Spark functionality. It represents the connection to a Spark cluster and can be used to create RDDs, accumulators, and broadcast variables on that cluster.

- SparkContext is the core of a Spark application and is responsible for coordinating the execution of tasks on the cluster.
- It manages the resources, acquires executors, and schedules tasks.

● Cluster Manager: SparkContext connects to the cluster manager, which is responsible for the allocation of resources (CPU, memory, etc.) in the cluster. The cluster manager can be Spark's standalone manager, Hadoop YARN, Mesos, or Kubernetes.

- Spark can run on various cluster managers, such as Apache Mesos, Hadoop YARN, or its built-in cluster manager.
- The cluster manager is responsible for resource allocation and job scheduling.
- The cluster manager is responsible for allocating resources to the Spark application. It can be Mesos, YARN, or Spark's standalone manager.
- It manages the distribution of application jars, resource allocation, and monitoring of cluster resources.

● Executors: Executors are worker nodes' processes in charge of running individual tasks in a given Spark job. They run concurrently across different nodes. Executors have two roles. Firstly, they run tasks that the driver sends. Secondly, they provide in-memory storage for RDDs.

- Executors are worker nodes in the Spark cluster. They run tasks as instructed by the SparkContext.
- Each application has its own set of executors, which are allocated by the cluster manager.

● Tasks: Tasks are the smallest unit of work in Spark. They are transformations applied to partitions. Each task works on a separate partition and is executed in a separate thread in executors.

● RDD: Resilient Distributed Datasets (RDD) are the fundamental data structures of Spark. They are an immutable distributed collection of objects, which can be processed in parallel. RDDs can be stored in memory between queries without the necessity for serialization.

- RDD is the fundamental data structure in Spark.
- RDD is a distributed, fault-tolerant collection of data that can be processed in parallel.
- RDDs are immutable and can be transformed and processed through operations like map, filter, reduce, etc.

● DAG (Directed Acyclic Graph): Spark represents a series of transformations on data as a DAG, which helps it optimize the execution plan. DAG enables pipelining of operations and provides a clear plan for task scheduling.

●DAG Scheduler: The Directed Acyclic Graph (DAG) Scheduler is responsible for dividing operator graphs into stages and sending tasks to the Task Scheduler. It translates the data transformations from the logical plan (which represents a sequence of transformations) into a physical execution plan. It optimizes the plan by rearranging and combining operations where possible, groups them into stages, and then submits the stages to the Task Scheduler.

- The DAG Scheduler is a high-level scheduler in Spark that manages the execution of stages within a Spark application. It works at the level of transformations and defines the logical execution plan.
- When you create a Spark application, it gets broken down into a series of stages. Each stage is a set of transformations that can be executed in parallel, and the stages are linked together to form a directed acyclic graph (DAG).

The DAG Scheduler's responsibilities include:

- Analyzing the logical execution plan: It examines the dependencies between stages and builds a DAG of stages.
- Optimizing the execution plan: It can reorganize stages to minimize data shuffling and improve performance.

- scheduling stages for execution: It schedules stages based on data availability and dependencies, and it tries to execute them as efficiently as possible.
- Handling stage failures: If a stage fails, the DAG Scheduler can determine which tasks need to be re-executed.
- The DAG Scheduler is responsible for making high-level decisions about how to execute a Spark application and is a critical component for optimizing Spark job execution.

● Task Scheduler: The Task Scheduler launches tasks via cluster manager. Tasks are the smallest unit of work in Spark, sent by the DAG Scheduler to the Task Scheduler. The Task Scheduler then launches the tasks on executor JVMs. Tasks for each stage are launched in as many parallel operations as there are partitions for the dataset.

- The Task Scheduler, also known as the Cluster Manager Scheduler, is a lower-level component of Spark responsible for managing the execution of individual tasks on worker nodes (executors).
- It interacts with the cluster manager (e.g., Mesos, YARN, or Spark's standalone manager) to acquire resources (CPU cores and memory) for task execution.
- Once resources are allocated, the Task Scheduler determines which tasks can be executed on which worker nodes based on data locality and other factors.
- The Task Scheduler launches tasks on executor nodes and monitors their progress. If a task fails, the Task Scheduler can request that it be re-executed on the same or different worker node, depending on the nature of the failure.
- The Task Scheduler ensures that tasks are evenly distributed across the cluster and that they make efficient use of available resources.
- It is responsible for reporting task status and resource usage back to the driver program.

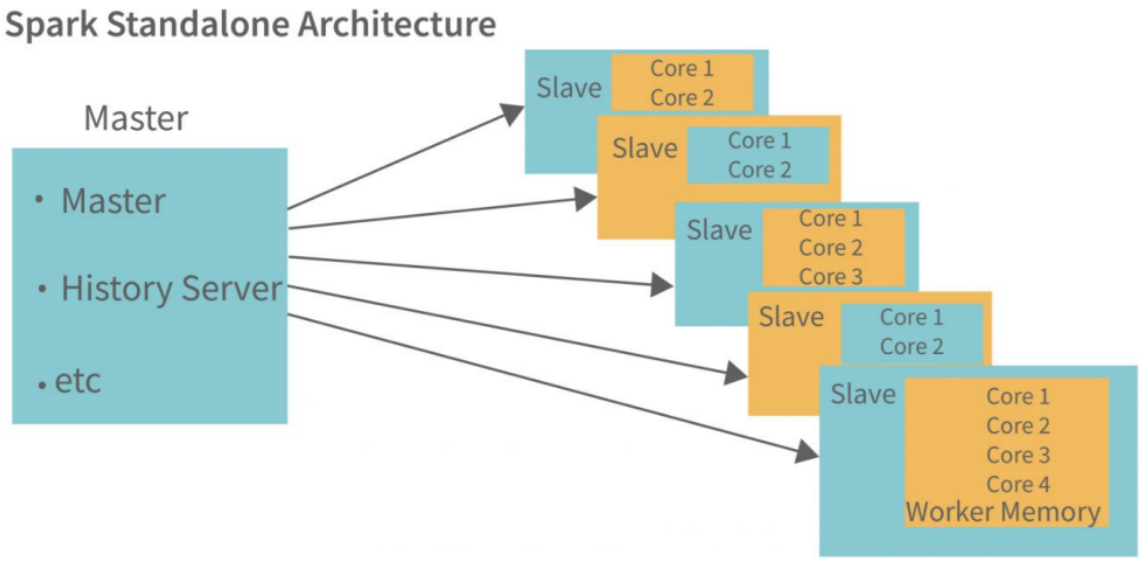
● Master: The Master is the base of a Spark Standalone cluster (specific to Spark's standalone mode, not applicable if Spark is running on YARN or Mesos). It's the central point and entry point of the Spark cluster. It is responsible for managing and distributing tasks to the workers. The Master communicates with each of the workers periodically to check if it is still alive and if it has completed tasks.

● Worker: The Worker is a node in the Spark Standalone cluster (specific to Spark's standalone mode). It receives tasks from the Master and executes them. Each worker has multiple executor JVMs running on it. It communicates with the Master and Executors to facilitate task execution. The worker is responsible for managing resources and providing an execution environment for the executor JVMs.

If Spark is running on YARN (Yet Another Resource Negotiator), the concept of a "master" node doesn't directly apply in the same way it does in Spark's standalone mode. Instead, resource management, scheduling and coordination are handled by YARN's own components.

……

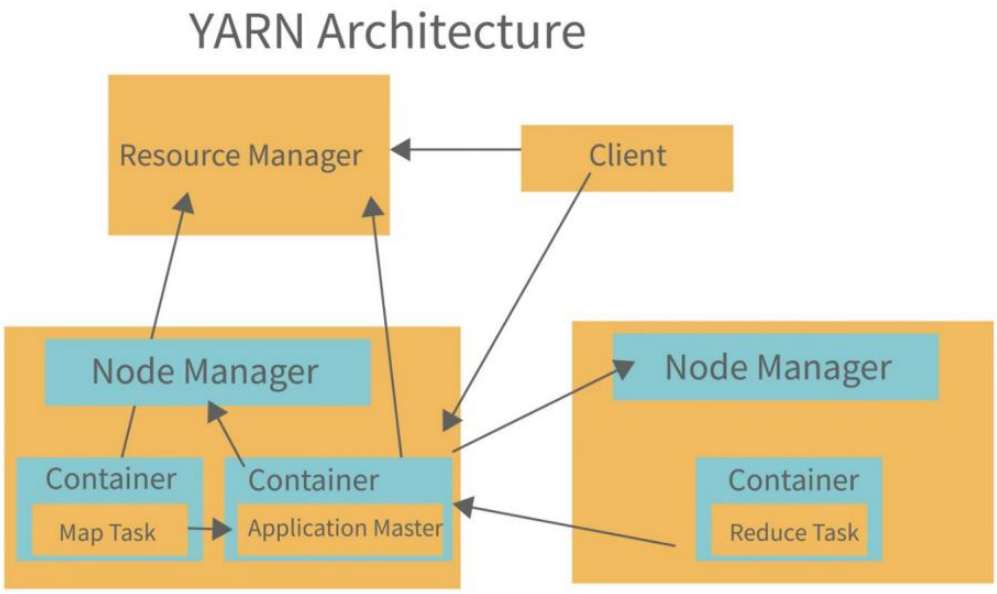
Spark With Standalone Cluster Manager Type:



- Standalone mode is Spark's built-in cluster manager, which allows you to deploy Spark on a cluster without the need for additional cluster management software.
- It is a simple, easy-to-set-up option for smaller clusters and can be used for both development and production deployments.
- While it lacks some of the advanced features of other cluster managers like Mesos or YARN, it works well for getting started with Spark.

……

Spark With YARN Cluster Manager Type:



● Resource Manager: It controls the allocation of system resources on all applications. A Scheduler and an Application Master are included. Applications receive resources from the Scheduler.

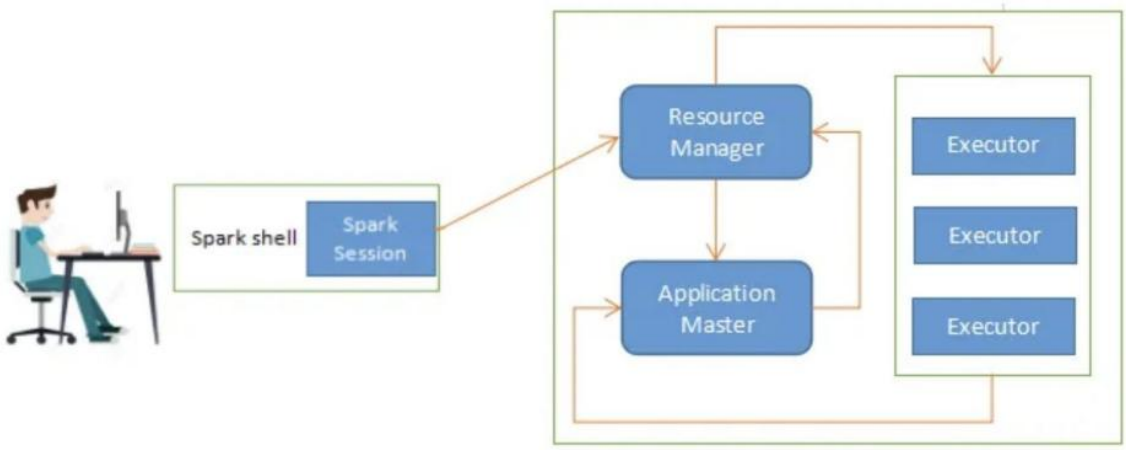
● Node Manager: Each job or application needs one or more containers, and the Node Manager monitors these containers and their usage. Node Manager consists of an Application Master and Container. The Node Manager monitors the containers and resource usage, and this is reported to the Resource Manager.

● Application Master: The Application Master (AM) is an instance of a framework-specific library and serves as the orchestrating process for an individual application in a distributed environment.

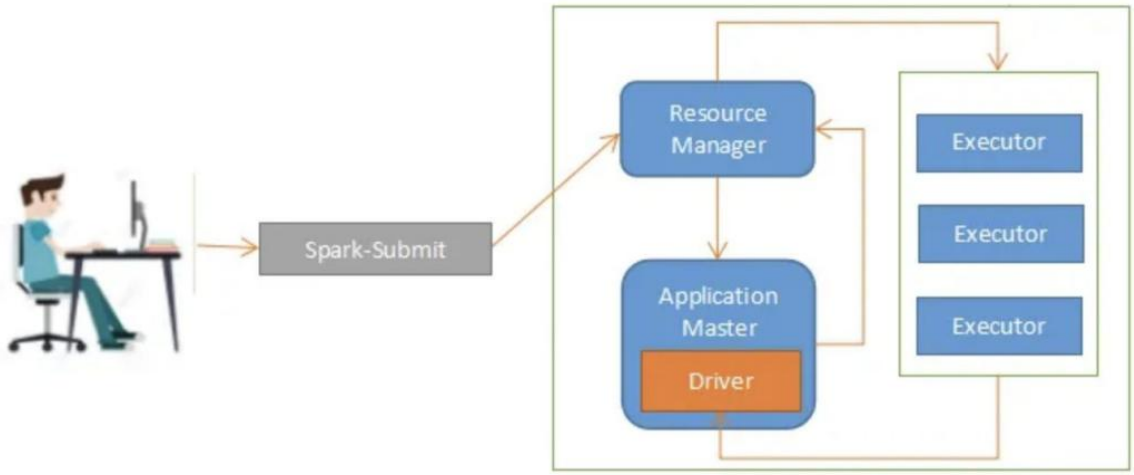
……

Deployment Modes Of Spark:

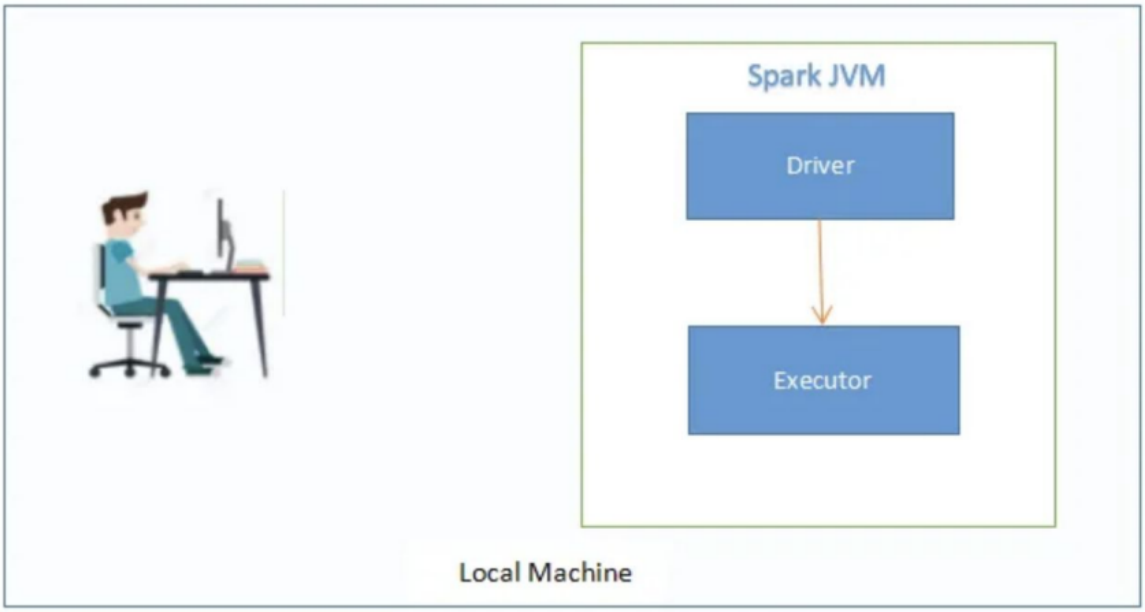
- Client Mode: When u start a spark shell, application driver creates the spark session in your local machine which request to Resource Manager present in cluster to create Yarn application. YARN Resource Manager start an Application Master (AM container). For client mode Application Master acts as the Executor launcher.
  Application Master will reach to Resource Manager and request for further containers. Resource manager will allocate new containers. These executors will directly communicate with Drivers which is present in the system in which you have submitted the spark application.



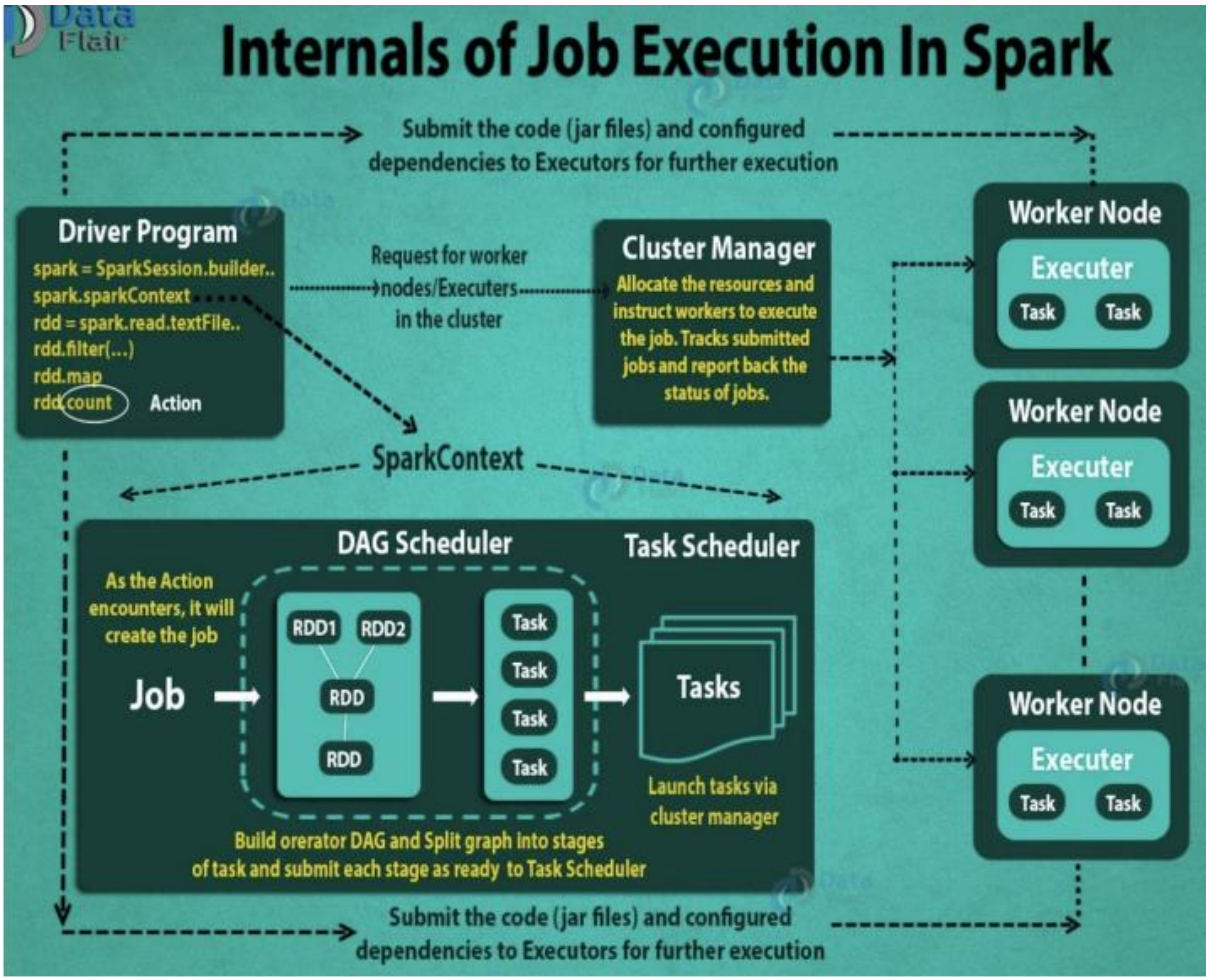
- Cluster Mode: For cluster mode, there's a small difference compare to client mode in place of driver. Here Application Master will create driver in it and driver will reach to Resource Manager.



- Local Mode: In local mode, Spark runs on a single machine, using all the cores of the machine. It is the simplest mode of deployment and is mostly used for testing and debugging.
- ➢ In local mode, Spark runs on a single machine, typically for development and testing purposes.
- ➢ It is suitable for running Spark on your local development machine, allowing you to test and prototype Spark applications without the need for a cluster setup.
- ➢ Local mode is not intended for processing large datasets or taking full advantage of Spark's distributed capabilities.



How Spark Job Runs Internally?



Apache Spark jobs are executed internally through a series of stages and tasks that are coordinated by the Spark engine. The internal execution process of a Spark job can be broken down into several steps:

- Job Submission:
  The Spark job begins with the submission of the application to a Spark cluster. This is typically done through the Spark driver program, which acts as the entry point for the application.
- Job Initialization:
  When the driver program is executed, it initializes a SparkContext. The SparkContext is responsible for coordinating the execution of the job and managing resources.
- Logical Plan Generation:
  The application defines a series of transformations on distributed data structures known as Resilient Distributed Datasets (RDDs).
  These transformations build a directed acyclic graph (DAG) that represents the logical plan of the job. The DAG describes the sequence of transformations that need to be applied to the input data.

- DAG Scheduler:
  The DAG Scheduler, a high-level scheduler, analyzes the logical execution plan (DAG) and optimizes it for efficient execution.
  It divides the plan into stages, where each stage consists of a set of transformations that can be executed in parallel.
- Task Generation:
  Once the stages are determined, the DAG Scheduler generates tasks for each stage. A task represents a single unit of work and typically corresponds to one partition of data.
  Tasks are created based on the RDD lineage, which describes the sequence of transformations needed to generate the data.
- Resource Allocation:
  The Spark job's tasks require resources, including CPU cores and memory. The cluster manager (e.g., Mesos, YARN, or Spark's standalone manager) is responsible for allocating these resources to the Spark application.
- Task Scheduling:
  The Task Scheduler, a lower-level scheduler, is responsible for scheduling the tasks for execution on available worker nodes (executors) based on data locality and resource availability.
  Tasks are sent to the worker nodes for execution.
- Task Execution:
  On each worker node, the assigned tasks are executed by the Spark executors. The data required for these tasks is fetched from other nodes as needed.
  Task execution includes applying the transformations defined in the logical plan and producing the desired output data.
- Shuffle and Data Exchange:
  In cases where data shuffling is required (e.g., during reduceByKey operations), Spark optimizes data exchange by minimizing the amount of data transferred over the network.
  Data is partitioned and grouped as efficiently as possible to reduce the data movement overhead.
- Result Collection (Actions):
  Once all tasks are completed, the results are collected and returned to the driver program.
  Actions, such as collect or count, trigger the execution of transformations and materialize the results.
- Job Completion:
  The job is considered complete when all tasks have finished successfully, and the driver program receives the final results.
  The driver program can then perform additional operations on the results, save the output, or take other desired actions.

Throughout this process, Spark ensures fault tolerance and reliability. In case of node failures or task failures, Spark can recover and re-execute only the necessary tasks by using information from the lineage of RDDs.

Overall, the execution of a Spark job involves the transformation of a logical plan into a series of tasks, resource allocation and scheduling, task execution, and the collection of results, all orchestrated by the Spark engine and its components.

……

Persist and Caching in Spark:

The methods persist() and cache() in Apache Spark are used to save the RDD, DataFrame, or Dataset in memory for faster access during computation. They are effectively ways to optimize the execution of your Spark jobs, especially when you have repeated transformations on the same data. However, they differ in how they handle the storage:

1. cache(): This method is a shorthand for using persist() with the default storage level. In other words, cache() is equivalent to calling persist() without any parameters.
   The default storage level is MEMORY_AND_DISK in PySpark and MEMORY_AND_DISK_SER in Scala Spark. This means that the RDD, DataFrame, or Dataset is stored in memory and, if it doesn't fit, the excess partitions are stored on disk.
   (((Cached memory in the context of Apache Spark refers to data that is stored in memory for fast access. When you cache data in Spark, it means you keep a copy of that data in memory, so it doesn't have to be read from the original source (like disk or a distributed file system) every time it's needed. This can significantly speed up data processing, especially for frequently accessed datasets.)))

2. persist(storageLevel): This method allows you to control how the data should be stored. You can pass a storage level as an argument to the persist() function, which gives you finer control over how the data is persisted.
   The storage level can be one of several options, each of which offers different trade-offs of memory usage and CPU efficiency and can use either memory or disk storage or both.

…….

Storage Levels in Persist:

- **StorageLevel.DISK_ONLY**: Store the RDD partitions only on disk.
- **StorageLevel.DISK_ONLY_2**: Same as the DISK_ONLY but replicate each partition on two cluster nodes.
- **StorageLevel.MEMORY_AND_DISK:** Store RDD as deserialized objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
- **StorageLevel.MEMORY_AND_DISK_2**: Like MEMORY_AND_DISK but replicate each partition on two cluster nodes.
- **StorageLevel.MEMORY_AND_DISK_SER:** Store RDD as serialized Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer, but more CPU-intensive to read.
- **StorageLevel.MEMORY_AND_DISK_SER_2:** Like MEMORY_AND_DISK_SER but replicate each partition on two cluster nodes.
- **StorageLevel.MEMORY_ONLY**: Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed.
- **StorageLevel.MEMORY_ONLY_2:** Like MEMORY_ONLY but replicate each partition on two cluster nodes.
- **StorageLevel.MEMORY_ONLY_SER**: Store RDD as serialized Java objects (one byte array per partition). This is more space-efficient, but more CPU-intensive to read.
- **StorageLevel.MEMORY_ONLY_SER_2**: Like MEMORY_ONLY_SER but replicate each partition on two cluster nodes.

How does data skewness occur in Spark?

Data skewness in Spark occurs when the data is not evenly distributed across partitions. This often happens when certain keys in your data have many more values than others. Consequently, tasks associated with these keys take much longer to run than others, which can lead to inefficient resource utilization and longer overall job execution time. Here are a few scenarios where data skewness can occur:

● Join Operations: When you perform a join operation on two datasets based on a key, and some keys have significantly more values than others, these keys end up having larger partitions. The tasks processing these larger partitions will take longer to complete.

● GroupBy Operations: Like join operations, when you perform a groupByKey or reduceByKey operation, and some keys have many more values than others, data skewness can occur.

● Data Distribution: If the data distribution is not uniform, such that certain partitions get more data than others, then data skewness can occur. This could happen due to the nature of the data itself or the partitioning function not distributing the data evenly.

……

How to deal with data skewness?

Handling data skewness is a common challenge in distributed computing frameworks like Apache Spark. Here are some popular techniques to mitigate it:

● Salting: Salting involves adding a random component to a skewed key to create additional unique keys. After performing the operation (like a join), the extra key can be dropped to get back to the original data.

- Identify Skewed Keys: First, you need to identify the keys that are causing data skewness. You can do this by monitoring your Spark application's logs or using profiling tools.
- Salting Logic: Create a salting logic that adds a random or unique prefix to the keys. The goal is to make previously skewed keys different from each other, so they are distributed across multiple partitions. You can use a function like scala.util.Random.nextInt to generate a random prefix.
- Repartition Data: After salting the keys, you need to repartition the data to distribute it more evenly across partitions. The number of partitions should be chosen to balance the workload and avoid creating too many partitions, which can lead to overhead.
- Remove Salting Prefix: When you are done with your processing, you may need to remove the salting prefix from the keys to obtain the original keys. You can do this using a transformation like the one used for salting, but in reverse.

● Splitting skewed data: Identify the skewed keys and process them separately. For instance, you can filter out the skewed keys and perform a separate operation on them.

● Increasing the number of partitions: Increasing the number of partitions can distribute the data more evenly. However, this might increase the overhead of managing more partitions.

● Using reduceByKey instead of groupByKey: reduceByKey performs local aggregation before shuffling the data, which reduces the data transferred over the network.

● Using Broadcast Variables: When joining a large DataFrame with a small DataFrame, you can use broadcast variables to send a copy of the small DataFrame to all nodes. This avoids shuffling of the large DataFrame.

……

Difference between Repartition & Coalesce in Spark:

- **repartition():**
  - This method is used to increase or decrease the number of partitions in an RDD or DataFrame.
  - It can cause a full shuffle of the data, i.e., all the data is reshuffled across the network to create new partitions.
  - This operation is expensive due to the full shuffle. However, the resulting partitions can be balanced, i.e., they have roughly equal amounts of data.
  - It can be used when you want to increase the number of partitions to allow for more concurrent tasks and increase parallelism when the cluster has more resources.
  - In certain scenarios, you may want to partition based on a specific key to optimize your job. For example, if you frequently filter by a certain key, you might want all records with the same key to be on the same partition to minimize data shuffling. In such cases, you can use repartition() with a column name.
- **coalesce():**
  - This method is used to reduce the number of partitions in an RDD or DataFrame.
  - It avoids a full shuffle. If you're decreasing the number of partitions, it will try to minimize the amount of data that's shuffled. Some partitions will be combined, and the data within these partitions will not need to be moved.
  - This operation is less expensive than repartition() because it minimizes data shuffling.
  - However, it can lead to data skew if you have fewer partitions than before, because it combines existing partitions to reduce the total number.

……

Example for Salting:

Imagine we have two DataFrames, df1 and df2, that we want to join on a column named 'id'. Assume that the 'id' column is highly skewed.

Firstly, without any handling of skewness, the join might look something like this:

```
result = df1.join(df2, on='id', how='inner')
```

Now, let's implement salting to handle the skewness:

```
import pyspark.sql.functions as F

# Define the number of keys you'll use for salting
num_salting_keys = 100

# Add a new column to df1 for salting
df1 = df1.withColumn('salted_key', (F.rand()*num_salting_keys).cast('int'))

# Explode df2 into multiple rows by creating new rows with salted keys
df2_exploded = df2.crossJoin(F.spark.range(num_salting_keys).withColumnRenamed('id', 'salted_key'))

# Now perform the join using both 'id' and 'salted_key'
result = df1.join(df2_exploded, on=['id', 'salted_key'], how='inner')

# If you wish, you can drop the 'salted_key' column after the join
result = result.drop('salted_key')
```

In this code, we've added a "salt" to the 'id' column in df1 and created new rows in df2 for each salt value. We then perform the join operation on both 'id' and the salted key. This helps to distribute the computation for the skewed keys more evenly across the cluster.

Difference between RDD, Dataframe and Dataset:

In Apache Spark, RDD (Resilient Distributed Dataset), DataFrame, and Dataset are all abstractions for working with distributed data, but they differ in their features and use cases. Here's a comparison of RDD, DataFrame, and Dataset:

- **RDD (Resilient Distributed Dataset):**

➢ RDD is the fundamental data structure in Spark, and it represents an immutable distributed collection of objects.
➢ RDDs are a low-level, more flexible, and general-purpose abstraction for distributed data processing.
➢ RDDs allow for both batch and real-time processing and can be used for low-level operations, such as map, filter, and reduce.
➢ RDDs provide no schema information, making them suitable for unstructured or semi-structured data.
➢ They offer strong type safety by allowing users to define their data structures explicitly.

- **DataFrame:**

- DataFrame is a higher-level abstraction built on top of RDD and introduced in Spark 1.3. It organizes data into named columns, much like a relational database or a spreadsheet.
- DataFrames have schema information, meaning that data is organized in a tabular format with named columns and data types.
- They support SQL-like queries using Spark SQL, making it easier to work with structured data.
- DataFrames are optimized for optimization and query planning using Spark's Catalyst optimizer. This makes them a better choice for complex analytics tasks.
- DataFrame API is available in multiple programming languages, including Scala, Java, Python, and R.

- **Dataset:**

- Dataset is a more recent addition to Spark, introduced in Spark 1.6. It can be thought of as a combination of the strengths of both RDD and DataFrame.
- Datasets are strongly typed, which means they offer the benefits of static typing while still providing the query optimization capabilities of DataFrames.
- They have schema information and can be queried using Spark SQL, similar to DataFrames.
- Datasets are designed for both batch and real-time processing, offering a more versatile and optimized option for a wide range of data processing tasks.
- Datasets are available in Scala and Java but are limited in Python and R due to the dynamic nature of these languages.

Here are some key considerations when choosing between RDD, DataFrame, and Dataset:

- Use RDD when you need fine-grained control, flexibility, and are working with unstructured or semi-structured data.
- Use DataFrames when working with structured data, need SQL-like querying capabilities, and benefit from query optimization.
- Use Datasets when you want the best of both worlds (strong typing and optimization) and have the option to work with structured or semi-structured data.

Criteria	RDD (Resilient Distributed Dataset)	DataFrame	DataSet
Abstraction	Low level, provides a basic and simple abstraction.	High level, built on top of RDDs. Provides a structured and tabular view on data.	High level, built on top of DataFrames. Provides a structured and strongly-typed view on data.
Type Safety	Provides compile-time type safety, since it is based on objects.	Doesn't provide compile-time type safety, as it deals with semi-structured data.	Provides compile-time type safety, as it deals with structured data.
Optimization	Optimization needs to be manually done by the developer (like using mapreduce).	Makes use of Catalyst Optimizer for optimization of query plans, leading to efficient execution.	Makes use of Catalyst Optimizer for optimization.
Processing Speed	Slower, as operations are not optimized.	Faster than RDDs due to optimization by Catalyst Optimizer.	Similar to DataFrame, it's faster due to Catalyst Optimizer.
Ease of Use	Less easy to use due to the need of manual optimization.	Easier to use than RDDs due to high-level abstraction and SQL-like syntax.	Similar to DataFrame, it provides SQL-like syntax which makes it easier to use.
Interoperability	Easy to convert to and from other types like DataFrame and DataSet.	Easy to convert to and from other types like RDD and DataSet.	Easy to convert to and from other types like DataFrame and RDD.

……

Understand Spark-Submit:

spark-submit is a command-line interface to submit your Spark applications to run on a cluster. It can use a number of supported master URLs to distribute your application across a cluster or can run the application locally. Here is the general structure of the spark-submit command:

```
spark-submit \
 --class <main-class> \
 --master <master-url> \
 --deploy-mode <deploy-mode> \
 --conf <key>=<value> \
 <application-jar> \
 [application-arguments]
```

--class: This is the entry point for your application, i.e., where your main method runs. For Java and Scala, this would be a fully qualified class name.

--master: This is the master URL for the cluster. It can be a URL for any Spark-supported cluster manager. For example,

local for local mode, spark://HOST:PORT for standalone mode, mesos://HOST:PORT for Mesos, or yarn for YARN.

Reference for list of configuration properties available in Spark 3.X -

https://spark.apache.org/docs/latest/configuration.html

--deploy-mode: This can be either client (default) or cluster. In client mode, the driver runs on the machine from which the job is submitted. In cluster mode, the framework launches the driver inside the cluster.

--conf: This is used to set any Spark property. For example, you can set Spark properties like spark.executor.memory, spark.driver.memory, etc.

<application-jar>: This is a path to your compiled Spark application.

[application-arguments]: These are arguments that you need to pass to your Spark application.

For example, if you have a Spark job written in Python and you want to run it on a local machine, your spark-submit command might look like this:

```
spark-submit \
 --master local[4] \
 --py-files /path/to/other/python/files \
 /path/to/your/python/application.py \
 arg1 arg2
```

In this example, --master local[4] means the job will run locally with 4 worker threads (essentially, 4 cores). --py-files is used to add .py, .zip or .egg files to be distributed with your application. Finally, arg1 and arg2 are arguments that will be passed to your Spark application.

```
spark-submit \
 --master spark://10.0.0.1:7077 \
 --deploy-mode client \
 --executor-memory 4G \
 --driver-memory 2G \
 --conf spark.app.name=WordCountApp \
```

```
--conf spark.executor.cores=2 \
--conf spark.executor.instances=5 \
--conf spark.default.parallelism=20 \
--conf spark.driver.maxResultSize=1G \
--conf spark.network.timeout=800 \
--py-files /path/to/other/python/files.zip \
/path/to/your/python/wordcount.py \
/path/to/input/textfile.txt
```

In this example:

--**master** spark://10.0.0.1:7077 tells Spark to connect to a standalone Spark cluster at the given IP address.

--**deploy-mode** client tells Spark to run the driver program on the machine that the job is submitted from, as opposed to within one of the worker nodes.

--**executor-memory 4G** and --**driver-memory 2G** set the amount of memory for the executor and driver processes, respectively.

--**conf spark.app.name=WordCountApp** sets a name for your application, which can be useful when monitoring your cluster.

--conf spark.executor.cores=2 sets the number of cores to use on each executor.

--conf spark.executor.instances=5 sets the number of executor instances

--conf spark.default.parallelism=20 sets the default number of partitions in RDDs returned by transformations like join(), reduceByKey(), and parallelize() when not set by user.

--conf spark.driver.maxResultSize=1G limits the total size of the serialized results of all partitions for each Spark action (e.g., collect). This should be at least as large as the largest object you want to collect.

--conf spark.network.timeout=800 sets the default network timeout value to 800 seconds. This configuration plays a vital role in cases where you deal with large shuffles.

--py-files /path/to/other/python/files.zip adds Python.zip, (. Egg) or .py files to distribute with your application. If you have Python files that your wordcount.py depends on, you can bundle them into a .zip or .egg file and pass it with --py-files.

/path/to/your/python/wordcount.py is the path to the Python file that contains your Spark application.

/path/to/input/textfile.txt is an argument that will be passed to your Spark application. In this case, it's a path to the text file that your WordCount application will process.

……

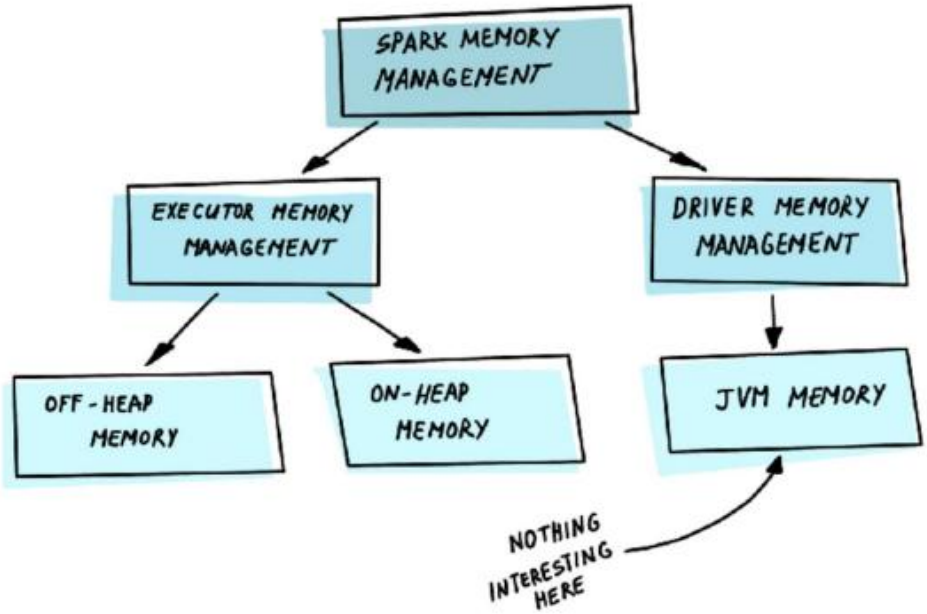
Memory Management in Spark:

When the Spark application is launched, the Spark cluster will start two processes — Driver and Executor.

The driver is a master process responsible for creating the Spark context, submission of Spark jobs, and translation of the whole Spark pipeline into computational units — tasks. It also coordinates task scheduling and orchestration on each Executor.

Driver memory management is not much different from the typical JVM process and therefore will not be discussed further.

The executor is responsible for performing specific computational tasks on the worker nodes and returning the results to the driver, as well as providing storage for RDDs. And its internal memory management is very interesting.



……

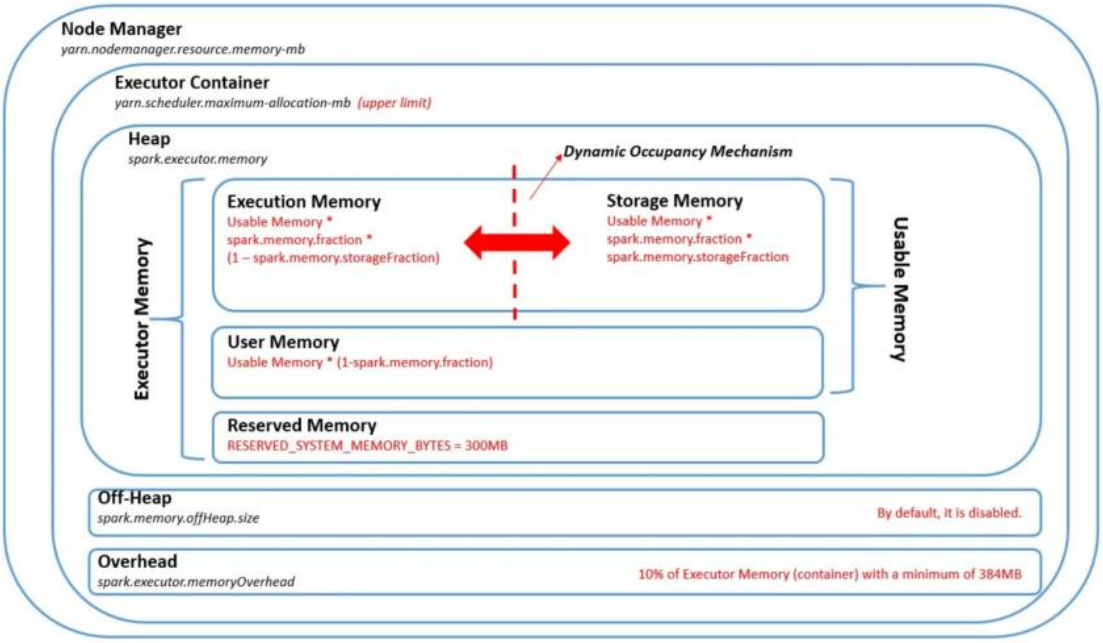
Memory Management - Executor Container:

When submitting a Spark job in a cluster with Yarn, Yarn allocates Executor containers to perform the job on different nodes.

ResourceManager handles memory requests and allocates executor container up to maximum allocation size settled by **yarn.scheduler.maximum-allocation-mb configuration**. Memory requests higher than the specified value will not take effect.

On a single node, it is done by **NodeManager**. NodeManager has an **upper limit** of resources available to it because it is limited by the resources of one node of the cluster. In Yarn it set up by **yarn.nodemanager.resource.memory-mb** configuration. It is the amount of physical memory per NodeManager, in MB, which can be allocated for yarn containers.

One **ExecutorContainer** is just one JVM. And the entire **ExecutorContainer** memory area is divided into three sections:



In a YARN cluster, you have multiple nodes, and each node is responsible for running tasks or containers. These nodes are managed by the YARN ResourceManager and NodeManager. The NodeManager is like a caretaker for an individual node in the cluster.

The NodeManager has a certain amount of physical memory available to it. Think of this memory like the RAM in your computer. This memory is used to run tasks or containers on that node.

The `**yarn.nodemanager.resource.memory-mb**` configuration setting is like a limit set on how much of that memory can be allocated for running tasks (containers). It's specified in megabytes (MB).

So, if you set `**yarn.nodemanager.resource.memory-mb**` to a certain value, say 8192 MB (which is 8 gigabytes), it means that each NodeManager can allocate up to 8 gigabytes of memory for running tasks. This is a limit set to ensure that the NodeManager doesn't use up all the memory on that node, leaving some for the operating system and other processes.

It's important to set this configuration properly because allocating too much memory to NodeManagers can lead to resource contention and poor performance, while allocating too little memory can cause tasks to fail or run very slowly. So, it's all about finding the right balance for your clusters.

{{{

To clear basics:

Yes, you've got the basic structure of a YARN (Yet Another Resource Negotiator) cluster right. Let's break it down:

1. **Cluster:**

- A YARN cluster consists of multiple machines (nodes) that work together to process and manage distributed applications.

2. **Node:**

- Each machine in the cluster is referred to as a "node."

- A node typically has its own CPU, memory, and other resources.

- These are individual machines or servers in a distributed computing cluster. Each node has its own CPU (core processor), RAM (memory), and potentially other hardware resources.

3.Executors**: These are processes that run on each node and are responsible for executing tasks as part of a distributed computing job. Executors utilize the CPU and RAM resources of their respective nodes to perform computations.**

CPU (Central Processing Unit): The CPU is the primary component of a computer that performs most of the processing. It executes instructions and carries out tasks on a computer. A CPU can consist of one or more core processors.

Core Processor: A core processor, often referred to simply as a "core," is an individual processing unit within a CPU. Many modern CPUs have multiple cores, which allows them to execute multiple tasks or threads simultaneously, thereby improving overall performance. Each core can independently execute instructions, and the more cores a CPU has, the better it can handle multitasking and parallel processing.

3. **NodeManager:**

- On each node, there is a component called the "NodeManager." It's responsible for managing resources and containers on that particular node.

- The NodeManager is like a caretaker for the resources on its node. It takes care of starting, monitoring, and stopping containers.

4. **Container:**

- A "container" is a unit of resource allocation in YARN. It represents a specific allocation of CPU and memory on a node.

- Applications (like MapReduce or Spark jobs) run within these containers. Each application is divided into one or more containers, and each container runs a specific task of the application.

So, the YARN cluster is made up of multiple nodes, and each node is managed by its NodeManager. Containers are used to allocate resources for running tasks within these nodes. The ResourceManager is another central component in the cluster that coordinates the allocation of resources across all nodes.

}}}

One ExecutorContainer is just one JVM. And the entire ExecutorContainer memory area is divided into three sections:

Sure, let's break down Spark memory management from nodes to executors and heap:

*Nodes:

- In a Spark cluster, you have multiple nodes, which are essentially individual machines or servers. These nodes work together to process data in a distributed manner.

*Executor (executer container):

- An executor is a process running on a node. Think of it as a worker on each machine in your cluster. Executors are responsible for running tasks and storing data in memory.

Executer container divide in 3:

a) **Heap –**

Executer memory – execution memory, storage memory, user memory and reserved memory

Usable memory - execution memory, storage memory, user memory

b) **off-heap**
c) **overhead**

**////////////

-It is mainly used to store temporary data in the shuffle, join, sort, aggregation, etc. Most likely, if your pipeline runs too long, the problem lies in the lack of space here.

Execution Memory = usableMemory * spark.memory.fraction * (1 - spark.memory.storageFraction)

-As Storage Memory, Execution Memory is also equal to 30% of all system memory by default (1 * 0.6 * (1 - 0.5) = 0.3).

2. **Storage Memory:**

- This is a portion of the heap where data is cached or stored in memory. For example, if you have frequently used datasets, they can be stored here for fast access.

-Storage Memory is used for caching and broadcasting data.

-Storage Memory size can be found by:

Storage Memory = usableMemory * spark.memory.fraction * spark.memory.storageFraction

-Storage Memory is 30% of all system memory by default (1 * 0.6 * 0.5 = 0.3).

{{

Note on cased-

Cached memory in the context of Apache Spark refers to data that is stored in memory for fast access. When you cache data in Spark, it means you keep a copy of that data in memory, so it doesn't have to be read from the original source (like disk or a distributed file system) every time it's needed. This can significantly speed up data processing, especially for frequently accessed datasets.

Here's a simple explanation with an example:

Imagine you have a large dataset of customer records, and you want to perform multiple operations on it using Spark. Instead of reading the data from a database or a file system every time you need it, you can cache it in memory. This means that the customer data is loaded into the memory of your Spark cluster's nodes.

Caching is useful when you have iterative algorithms or when you need to reuse the same dataset in multiple Spark transformations. However, you should be mindful of the available memory in your cluster because caching too much data can lead to out-of-memory errors. You can also explicitly uncache data when you no longer need it to free up memory resources.

}}}

In the context of Apache Spark, there are two key memory components that you need to understand: Reserved Memory and User Memory.

3. **Reserved Memory:**

- Reserved memory, is a portion of memory that Spark sets aside for internal bookkeeping and metadata. Its memory used by Spark for tasks like managing data structures, tracking job progress, and handling the system's needs.

- Reserved memory is not available for user-level operations or caching data. It's purely for Spark's internal use.

- You can configure the amount of reserved memory in Spark using the `spark.memory.offHeap.size` configuration property. By default, this is set to 300MB.

- In the test environment (when spark.testing set) we can modify it with spark.testing.reservedMemory.

4. **User Memory:**

The explanation you've provided is accurate. User memory in Apache Spark is indeed a memory region where you can store your own data structures, or any data needed for RDD (Resilient Distributed Dataset) transformations. It's a region where you can manage your own data and calculations without Spark's direct oversight. Here's a breakdown of the key points:

- **Usage for RDD Conversion and Lineage:**

- User memory is typically used to store data structures required for RDD conversion operations and maintaining lineage information. Lineage information is crucial for ensuring fault tolerance and recomputing data in case of node failures.

- **Custom Data Structures:**

- You have the flexibility to store your own custom data structures or variables in this memory space. For example, if you have some intermediate results or data that need to be shared among various transformations, you can store them in user memory.

- **No Strict Accounting by Spark:**

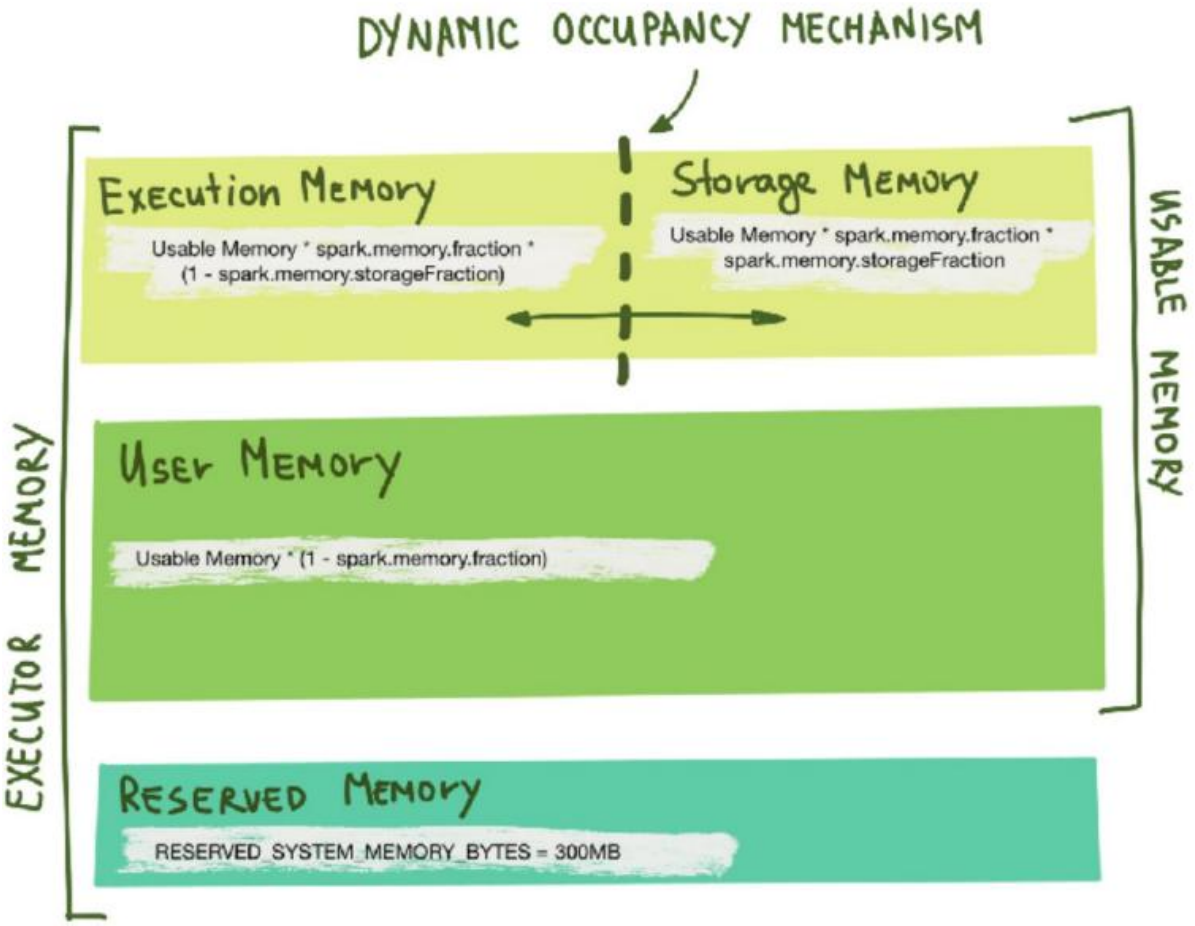
- Spark does not enforce strict accounting or management of user memory. It's your responsibility to manage and control what data is placed in user memory and how it's utilized.

- **Boundary Responsibility:**

- While Spark doesn't directly regulate or monitor user memory, it's important to be responsible and mindful of how you use this memory region. Mismanaging it can lead to performance issues, out-of-memory errors, or inefficient memory utilization.

User memory provides a level of freedom for developers to work with their own data structures and variables within the Spark framework. However, with this flexibility comes the responsibility to use this memory wisely to ensure the efficient and reliable execution of Spark applications.

- **It is mainly used to store data needed for RDD conversion operations, such as lineage. You can store your own data structures there that will be used inside transformations. It's up to you what would be stored in this memory and how.**
- **Spark makes completely no accounting on what you do there and whether you respect this boundary or not.**
- **User Memory = usableMemory * (1 -spark.memory.fraction) It is 1 * (1 - 0.6) = 0.4 or 40% of available memory by default.**



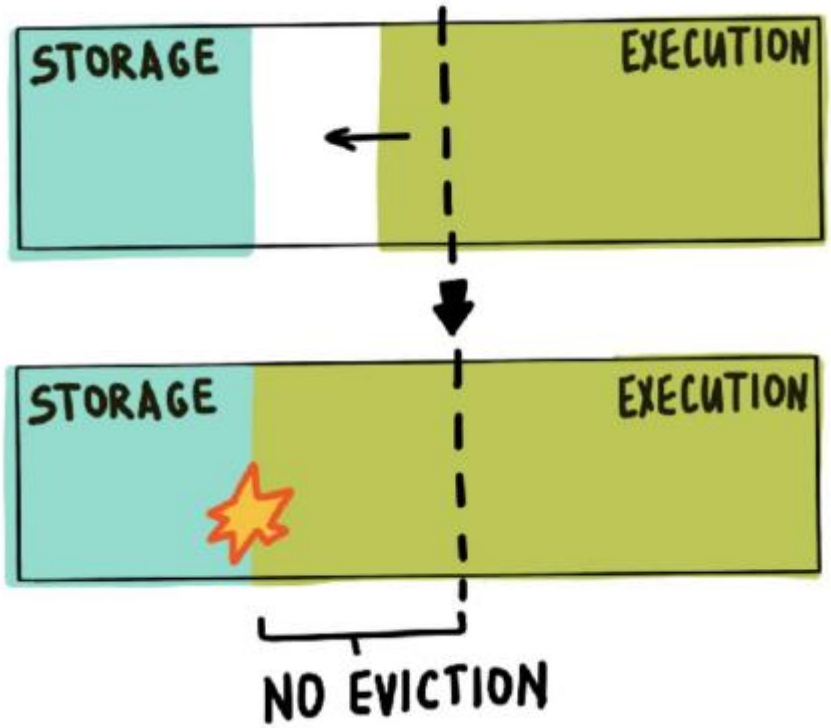
////////////

- **Spark's core transformations and actions do not use off-heap memory by default. This can be configured using spark.executor.memoryOffHeap.enabled and spark.executor.memoryOffHeap.size**
- **Spark may serialize data structures, such as RDD partitions, and store them in off-heap memory. This reduces the risk of garbage collection (GC) overhead and allows for more efficient memory management. Serialized data is stored outside the JVM's heap space.**

**////////////

Dynamic Occupancy Mechanism:

Execution and Storage have a shared memory. They can borrow it from each other. This process is called the Dynamic occupancy mechanism.



There are two parts of the shared memory — the Storage side and the Execution side. The shared Storage memory can be used up to a certain threshold. In the code, this threshold is called onHeapStorageRegionSize. This part of memory is used by Storage memory, but only if it is not occupied by Execution memory. Storage memory has to wait for the used memory to be released by the executor processes. The default size of onHeapStorageRegionSize is all Storage Memory.

////////////

When Execution memory is not used, Storage can borrow as much Execution memory as available until execution reclaims its space. When this happens, cached blocks will be evicted from memory until sufficient borrowed memory is released to satisfy the Execution memory request.

The creators of this mechanism decided that Execution memory has priority over Storage memory. They had reasons to do so — the execution of the task is more important than the cached data, the whole job can crash if there is an OOM in the execution.

……

Memory understanding:

- Bit: 1 Bit = 1/8 Byte
- Byte: 8 Bits = 1 Byte
- Kilobyte (KB): 1024 Bytes = 1,024 Bytes
- Megabyte (MB): 1,024 Kilobytes = 1,048,576 Bytes
- Gigabyte (GB): 1,024 Megabytes = 1,073,741,824 Bytes
- Terabyte (TB): 1,024 Gigabytes = 1,099,511,627,776 Bytes
- Petabyte (PB): 1,024 Terabytes = 1,125,899,906,842,624 Bytes
- Exabyte (EB): 1,024 Petabytes = 1,152,921,504,606,846,976 Bytes
- Zettabyte (ZB): 1,024 Exabytes = 1,180,591,620,717,411,303,424 Bytes
- Yottabyte (YB): 1,024 Zettabytes = 1,208,925,819,614,629,174,706,176 Bytes

……

{{{

note :

1. **The 1 executer always for application master yarn and then you must give remaining for the spark submit job for final.**
   Ex- 20 executers so 1 for yarn (application master) and then you provide final as 19executer to spark job.

2.

on each node, 1 core and 1 GB is needed for Operating System and Hadoop Daemons. The 1 GB of RAM and 1 core reserved for the operating system and Hadoop daemons is typically a shared resource for the entire node.

…..you can say for each node or for out of overall gb.

Ex – 64 gb so 1 gb reserve for os and Hadoop demon always so final used is 63 gb.

2. **It is better to give 5 core to each executers if you give more so it perform bad as research shows.**

Calculate overhead memory:

Overhead = max (384 mb,0.07*spark.executor.memory)

…means if my answer is less than 384 mb so I consider my overhead memory as 384mb.

If it would be greater than 384mb after calculation, then the greater one considers as the overhead memory.

Formulas:

We know that now how much memory is per executer.

So, we need to calculate:

1. **Reserved Memory:**
   So reserved memory is always 300 mb per executer.
   If you want to find for all executer so 300* no.of executers.

2. **Storage Memory:**
   So, storage memory is always 30 % of executer memory.

   Storage Memory = usableMemory * spark.memory.fraction * spark.memory.storageFraction

   (((ex- useble memory means see diagram ……
   It a memory that is left memory after subtracting the reserved memory from total memory per executer.

   So, suppose 1gb is total memory per executer so
   Reserved memory is always – 300 mb.
   Usable memory = 1gb – 300 mb
   Usable memory = 1024-300 = 724
   )))

   (((spark fraction is fix 0.6 and memory storage fraction is also fix 0.5 they fix but you can change them according to scenario

   Understand in better –

   spark.memory.fraction –

   it is the hip memory space allocated to spark execution memory.

   The value is expressed as a double between 0 and 1, where 0.6, for example, would mean 60% of the heap is allocated for execution memory.

   …… he ek execution memory ahe

spark.memory.storageFraction :

it is the hip memory space allocated to spark storage memory.

The value is expressed as a double between 0 and 1, where 0.5, for example, would mean 50% of the heap is allocated for storage memory.

…… he ek storage memory ahe

The combination of these two parameters helps control the trade-off between execution memory (used for tasks like shuffling and sorting) and storage memory (used for caching data in-memory). Adjusting these values can impact the performance of your Spark application depending on your specific workload and hardware.

Ex- Large-Scale Data Processing:

If your Spark application processes very large datasets, you may need to allocate more memory to execution memory (spark.memory.fraction) to accommodate the processing of data, shuffling, and sorting.

Caching and Iterative Algorithms:

Applications that use caching extensively or employ iterative algorithms may require a higher spark.memory.storageFraction to allocate more memory for caching data in memory.

)))

So, storage memory = 724*0.6*0.5 = 217mb

But suppose in other cases it would more executer memory and less storage memory required so it change.

3. Execution Memory:

So, storage memory is also around 30 % of executer memory.

Execution Memory = usableMemory * spark.memory.fraction * (1 - spark.memory.storageFraction)

(((Ex-
Execution memory = 724*0.6(1-0.5)
Execution memory = 217 mb

3. **User Memory:**

   40% of available memory by default.

User Memory = usableMemory * (1 - spark.memory.fraction)

User Memory = 724*(1-0.6)

User Memory = 290 mb

……

How to process 1 TB of data in Apache Spark?

Total blocks - 128MB default block size (HDFS) - 1TB -> 1 *1024 1024/128 = 8192.

Let us take

-> 20 Executor-machine

-> 5 Core processor in each executor node

-> 6 GB RAM in each executor node

And the cluster can perform (20*5=100) Task parallel at a time, Here tasks mean block so 100 blocks can be processed

parallelly at a time.

100*128MB = 12800 MB / 1024GB = 12.5 GB (So, 12GB data will get processed in 1st set of a batch)

Since the RAM size is 6GB in each executor, (20 executor x 6GB RAM =120GB Total RAM) So, at a time 12GB of RAM

will occupy in a cluster (20node/12gb=1.6GB RAM In each executor).

Now, Available RAM in each executor will be (6GB - 1.6GB = 4.4GB) RAM which will be reserved for other users' jobs

and programs.

So, 1TB = 1024 GB / 12GB = (Whole data will get processed in around 85 batches).

Note :- Actual values may differ in comparison with real-time scenarios.

……

Case studies for resource static allocation:

We start with how to choose the number of cores:

Number of cores = Concurrent tasks an executor can run

So we might think, more concurrent tasks for each executor will give better performance. But research shows that any application with more than 5 concurrent tasks, would lead to a bad show. So the optimal value is 5.

This number comes from the ability of an executor to run parallel tasks and not from how many cores a system has. So the number 5 stays the same even if we have double (32) cores in the CPU

- 5 cores per executor
  - For max HDFS throughput
- Cluster has 6 * 15 = 90 cores in total (after taking out Hadoop/Yarn daemon cores)
- 90 cores / 5 cores/executor = 18 executors
- 1 executor for AM => 17 executors
- Each node has 3 executors
- 63 GB/3 = 21 GB, 21 x (1-0.07) ~ 19 GB (counting off heap overhead)

Number of executors:

Coming to the next step, with 5 as cores per executor, and 15 as total available cores in one node (CPU) — we come to 3 executors per node.

which is 15/5. We need to calculate the number of executors on each node and then get the total number for the job.

So, with 6 nodes and 3 executors per node — we get a total of 18 executors. Out of 18, we need 1 executor (java process) for Application Master in YARN. So, the final number is 17 executors.

This 17 is the number we give to spark using –num-executors while running from the spark-submit shell command.

Memory for each executor:

From the above step, we have 3 executors per node. And available RAM on each node is 63 GB.

So, memory for each executor in each node is 63/3 = 21GB.

However small overhead memory is also needed to determine the full memory request to YARN for each executor.

The formula for that overhead is max(384, .07 * spark.executor.memory) , Calculating that

overhead: 0.07 * 21 (Here 21 is calculated as above 63/3) = 1.47

Since 1.47 GB > 384 MB, the overhead is 1.47

Take the above from each 21 above => 21–1.47 ~ 19 GB , So executor memory — 19 GB

Final numbers — Executors — 17, Cores 5, Executor Memory — 19 GB

……

Case 2 Hardware — 6 Nodes and Each node have 32 Cores, 64 GB:

Number of cores of 5 is the same for good concurrency as explained above.

Number of executors for each node = 32/5 ~ 6

So total executors = 6 * 6 Nodes = 36. Then the final number is 36–1(for AM) = 35

Executor memory:

6 executors for each node. 63/6 ~ 10. Overhead is .07 * 10 = 700 MB. So rounding to 1GB as overhead, we get 10–1 = 9 GB

Final numbers — Executors — 35, Cores 5, Executor Memory — 9 GB

……

Case 3 — When more memory is not required for the executors:

The above scenarios start with accepting the number of cores as fixed and moving to the number of executors and memory.

Now for the first case, if we think we do not need 19 GB, and just 10 GB is sufficient based on the data size and computations involved, then following are the numbers:

Cores: 5

Number of executors for each node = 3. Still, 15/5 as calculated above.

At this stage, this would lead to 21 GB, and then 19 as per our first calculation. But since we thought 10 is ok (assume little overhead), then we cannot switch the number of executors per node to 6 (like 63/10). Because with 6 executors per node and 5 cores it comes down to 30 cores per node when we only have 16 cores. So, we also need to change the number of cores for each executor.

So, calculating again,

The magic number 5 comes to 3 (any number less than or equal to 5). So, with 3 cores, and 15 available cores — we get 5 executors per node, 29 executors (which is (5*6 -1)) and memory is 63/5 ~ 12.

The overhead is 12*.07=.84. So, executor memory is 12–1 GB = 11 GB

Final Numbers are 29 executors, 3 cores, executor memory is 11 GB.

……

Broadcast and Accumulators in Spark:

In Spark, broadcast variables are read-only shared variables that are cached on each worker node rather than sent over the network with tasks. They're used to give every node a copy of a large input dataset in an efficient manner. Spark's actions are executed through a set of stages, separated by distributed "shuffle" operations. Spark automatically broadcasts the common data needed by tasks within each stage.

To create a broadcast variable in Spark, you can use the **SparkContext.broadcast().**

```
from pyspark import SparkContext

sc = SparkContext("local", "BroadcastExample")
data = [1, 2, 3, 4, 5]
broadcast_data = sc.broadcast(data)

def process_element(x):
    return x * broadcast_data.value

rdd = sc.parallelize([10, 20, 30, 40, 50])
result = rdd.map(process_element)
print(result.collect())
```

Accumulators:

Accumulators are write-only variables that are used to accumulate results across multiple tasks in a parallel or distributed computation. They are typically used for aggregating data or collecting statistics during Spark transformations, such as “map “or “reduce”, without the need for synchronization between worker nodes. Accumulators can only be updated in an associative and commutative manner and are used for tasks like counting elements, summing values, or accumulating statistics.

To create an accumulator in Spark, you can use the **SparkContext.accumulator()**

```python
from pyspark import SparkContext

sc = SparkContext("local", "AccumulatorExample")
accumulator = sc.accumulator(0)

def process_element(x):
    global accumulator
    accumulator += x
    return x

rdd = sc.parallelize([10, 20, 30, 40, 50])
rdd.foreach(process_element)
print("Accumulated value:", accumulator.value)
```

.....

Handling Failures in Spark:

Driver Failure:

The driver program runs the main() function of the application and creates a SparkContext. If the driver node fails, the entire application will be terminated, as it's the driver program that declares transformations and actions on data and submits such requests to the cluster.

Impact:

● The driver node is a single point of failure for a Spark application.

● If the driver program fails due to an exception in user code, the entire Spark application is terminated, and all executors are released.

Handling Driver Failure:

● Driver failure is usually fatal, causing the termination of the application.

● It's crucial to handle exceptions in your driver program to prevent such failures.

● Also, monitor the health of the machine hosting the driver program to prevent failures due to machine errors.

● In some cluster managers like Kubernetes, Spark supports mode like **spark.driver.supervise** to supervise and restart the driver on failure.

Executor Failure:

Executors in Spark are responsible for executing the tasks. When an executor fails, the tasks that were running will fail.

Impact:

● Executors can fail for various reasons, such as machine errors or OOM errors in the user's application.

● If an executor fails, the tasks that were running on it are lost.

● The failure of an executor doesn't cause the failure of the Spark application, unless all executors fail.

Handling Executor Failure:

● If an executor fails, Spark can reschedule the failed tasks on other executors.

● There is a certain threshold for task failures. If the same task fails more than 4 times (default), the application will be terminated.

● Make sure to tune the resources allocated for each executor, as an executor might fail due to insufficient resources.

● For resilience, you can also opt to replicate the data across different executor nodes.

……..

Failure Due to Out Of Memory in Spark:

Spark Driver OOM Scenarios:

● Large Collect Operations: If the data collected from executors using actions such as collect() or take() is too large to fit into the driver's memory, an OutOfMemoryError will occur.

○ Solution: Be cautious with actions that pull large volumes of data into the driver program. Use actions like take(n), first(), collect() carefully, and only when the returned data is manageable by the driver.

● Large Broadcast Variables: If a broadcast variable is larger than the amount of free memory on the driver node, this will also cause an OOM error.

○ Solution: Avoid broadcasting large variables. If possible, consider broadcasting a common subset of the data, or use Spark's built-in broadcast join if joining with a large DataFrame.

● Improper Driver Memory Configuration: If spark.driver.memory is set to a high value, it can cause the driver to request more memory than what is available, leading to an OOM error.

○ Solution: Set the spark.driver.memory config based on your application's need and ensure it doesn't exceed the physical memory limits.

Spark Executor OOM Scenarios:

● Large Task Results: If the result of a single task is larger than the amount of free memory on the executor node, an OutOfMemoryError will occur.

○ Solution: Avoid generating large task results. This is often due to a large map operation. Consider using reduceByKey or aggregateByKey instead of groupByKey when transforming data.

● Large RDD or DataFrame operations: Certain operations on RDDs or DataFrames, like join, groupByKey, reduceByKey, can cause data to be shuffled around, leading to a large amount of data being held in memory at once, potentially causing an OOM error.

○ Solution: Be cautious with operations that require shuffling large amounts of data. Use operations that reduce the volume of shuffled data, such as reduceByKey and aggregateByKey, instead of groupByKey.

● Persistent RDDs/DataFrames: If you're persisting many RDDs/DataFrames in memory and there isn't enough memory to store them, this will also cause an OOM error.

○ Solution: Unpersist unnecessary RDDs and DataFrames as soon as they are no longer needed. Tune the spark.memory.storageFraction to increase the amount of memory reserved for cached RDDs/DataFrames.

● Improper Executor Memory Configuration: Similar to the driver, if spark.executor.memory is set to a high value, it can cause the executor to request more memory than what is available, leading to an OOM error.

○ Solution: Set the spark.executor.memory config based on your application's need and ensure it doesn't exceed the physical memory limits of the executor nodes.

……..

Code Level and Resource Level Optimizations in Spark:

Code Level Optimization:

● Use DataFrames/Datasets instead of RDDs: DataFrames and Datasets have optimized execution plans, leading to faster and more memory-efficient operations than RDDs. They also have more intuitive APIs for many operations.

● Leverage Broadcasting: If you're performing an operation like a join between a large DataFrame and a small DataFrame, consider broadcasting the smaller DataFrame. Broadcasting sends the smaller DataFrame to all worker nodes, so they have a local copy and don't need to fetch the data across the network.

● Avoid Shuffling: Operations like groupByKey cause shuffling, where data is transferred across the network, which can be slow. Operations like reduceByKey or aggregateByKey reduce the amount of data that needs to be shuffled, and can be faster.

● Avoid Collecting Large Data: Be careful with operations like collect() that bring a large amount of data into the driver program, which could cause an out of memory error.

● Repartitioning and Coalescing: Depending on your use case, you might want to increase or decrease the number of partitions. If you have too many small partitions, use coalesce to combine them. If you have too few large partitions, use repartition to split them.

● Persist/Cache Wisely: Persist or cache the DataFrames or RDDs that you'll reuse. However, keep in mind that these operations consume memory, so use them judiciously.

Resource Configuration Optimization:

● Tune Memory Parameters: Make sure to set spark.driver.memory, spark.executor.memory, spark.memory.fraction, and spark.memory.storageFraction based on the memory requirements of your application and the capacity of your hardware.

● Control Parallelism: Use spark.default.parallelism and spark.sql.shuffle.partitions to control the number of tasks during operations like join, reduceByKey, etc. Too many tasks can cause a lot of overhead, but too few tasks might not fully utilize your cluster.

● Dynamic Allocation: If your cluster manager supports it, use dynamic resource allocation, which allows Spark to dynamically adjust the resources your application occupies based on the workload. This means that if your application has stages that require lots of resources, they can be allocated dynamically.

spark.dynamicAllocation.enabled true

spark.dynamicAllocation.initialExecutors 2

spark.dynamicAllocation.minExecutors 1

spark.dynamicAllocation.maxExecutors 20

spark.dynamicAllocation.schedulerBacklogTimeout 1m

spark.dynamicAllocation.sustainedSchedulerBacklogTimeout 2m

spark.dynamicAllocation.executorIdleTimeout 2min

❖ spark.dynamicAllocation.enabled is set to true to enable dynamic allocation.

❖ spark.dynamicAllocation.initialExecutors is set to 2 to specify that initially, two executors will be allocated.

❖ spark.dynamicAllocation.minExecutors and spark.dynamicAllocation.maxExecutors control the minimum and maximum number of executors, respectively.

❖ spark.dynamicAllocation.schedulerBacklogTimeout and spark.dynamicAllocation.sustainedSchedulerBacklogTimeout control how long a backlog of tasks Spark will tolerate before adding more executors.

❖ spark.dynamicAllocation.executorIdleTimeout controls how long an executor can be idle before Spark removes it.

● Tune Garbage Collection: Spark uses the JVM, so the garbage collector can significantly affect performance. You can use spark.executor.extraJavaOptions to pass options to the JVM to tune the garbage collection.

● Use Appropriate Data Structures: Parquet and Avro are both columnar data formats that are great for analytical queries and schema evolution. If your data processing patterns match these, consider using these formats.

……

Best practices to design Spark application:

Understand Your Data and Workload:

● Data Size: Understand the volume of data your application will process. This will influence the infrastructure needed and the parallelism level in your application.

● Data Skewness: Identify if your data is skewed, as this can cause certain tasks to take longer to complete and negatively impact performance. Techniques like key salting can be applied to handle skewness.

● Workload Type: Understand the type of operations your application will perform. For example, analytical workloads may benefit from columnar data formats like Parquet or ORC.

Application Design:

● Transformations: Use transformations like map, filter, reduceByKey over actions as much as possible, as transformations are lazily evaluated and can benefit from Spark's optimization.

● Shuffling: Minimize operations that cause data shuffling across the network, as they are expensive. Avoid operations like groupByKey in favor of reduceByKey or aggregateByKey.

● Broadcasting: Small datasets that are used across transformations should be broadcasted to improve performance.

● Partitioning: Use the right level of partitioning. Too few partitions can lead to fewer concurrent tasks and underutilization of resources. Too many partitions might lead to excessive overhead.

Resource Allocation and Configuration:

● Memory Allocation: Properly allocate memory to Spark executors, driver, and overhead. Monitor the memory usage of your application to fine-tune these settings.

● Dynamic Resource Allocation: Enable dynamic resource allocation if supported by your cluster manager. This allows Spark to adjust the resources based on workload.

● Parallelism: Configure the level of parallelism (number of partitions) based on the data volume and infrastructure.Infrastructure Consideration:

● Storage: Use fast storage (like SSDs) to store your data, as I/O operations can become a bottleneck in large data processing.

● Network: A high-speed network is important, especially if your workload involves data shuffling.

● Nodes and Cores: More nodes with multiple cores can increase parallelism and data processing speed.

● Data Locality: Aim for data locality, i.e., running tasks on nodes where the data is stored, to reduce network I/O.

● Use Spark's built-in web UI to monitor your applications. Look for stages that take a long time to complete, tasks that fail and are retried, storage and computation bottlenecks, and executor memory usage.

……

Spark notes 4 and spark notes _1.pdf baki 1st 2 complete ahet